



TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN COMPUTACIÓN



Diseño e implementación de un sistema de navegación volumétrica para agentes voladores en videojuegos

Estudiante: Javier Manotas Ruiz
Dirección: Enrique Fernández Blanco
Alejandro Puente Castro

A Coruña, xuño de 2026.

Índice general

1	Introducción	1
1.1	Motivación	2
1.2	Objetivos	4
1.3	Estructura de la memoria	5
2	Fundamentos	6
2.1	Fundamentos tecnológicos	6
2.1.1	Unity y su modelo de componentes	6
2.1.2	Herramientas proporcionadas por el editor	7
2.1.3	Motor de físicas y consultas espaciales	8
2.1.4	Data-Oriented Technology Stack	8
2.1.5	Características de bajo nivel en C#	9
2.2	Herramientas y entorno de desarrollo	10
2.3	Fundamentos teóricos	11
2.3.1	Búsqueda de caminos	11
2.3.2	Planificación de caminos	14
2.3.3	Integridad estructural mediante algoritmos Hash	16
3	Estado de la cuestión	17
3.1	Herramientas disponibles	17
3.1.1	Herramientas genéricas	17
3.1.2	Soluciones específicas de motor	18
3.2	Comparativa de mercado	20
3.2.1	Justificación del entorno de desarrollo	20
3.3	DAFO	21
3.3.1	Análisis interno	21
3.3.2	Análisis externo	21
3.3.3	Conclusiones	22

4	Metodología y planificación	23
4.1	Metodología de desarrollo	23
4.2	Planificación inicial	24
4.2.1	Planificación temporal	24
4.2.2	Estimación de costes	25
4.3	Seguimiento	25
5	Desarrollo	28
5.1	Iteración 0: Cimientos del proyecto	28
5.2	Iteración 1: Representación del espacio navegable	30
5.2.1	Organización jerárquica	30
5.2.2	Codificación de las hojas mediante manipulación de bits	30
5.2.3	Códigos de Morton	31
5.2.4	Punteros compactos: el enlace del SVO	31
5.3	Iteración 2: Construcción y rasterización del volumen	32
5.3.1	Muestreo del espacio mediante consultas físicas paralelizadas	32
5.3.2	Optimización: descarte de hojas vacías	33
5.3.3	Enlazado de la vecindad	35
5.4	Iteración 3: Búsqueda de rutas tridimensional	35
5.4.1	Búsqueda sobre un grafo de resolución heterogénea	35
5.4.2	Función de coste y heurística	37
5.4.3	Reconstrucción y robustez	37
5.5	Iteración 4: Postprocesado de trayectorias	37
5.5.1	Atajo voraz mediante línea de visión	37
5.5.2	Suavizado mediante <i>splines</i> de Catmull-Rom	38
5.6	Iteración 5: Persistencia e integridad de la escena	39
5.6.1	Serialización en un único bloque binario	39
5.6.2	Integridad de la escena: reinterpretación de los bits de un <i>float</i>	39
5.7	Iteración 6: Robustez geométrica y radio del agente	40
5.7.1	Consideración del radio del agente	40
5.7.2	Validación intensiva mediante pruebas	41
5.8	Iteración 7: Integración en Unity y telemetría	41
5.8.1	Componentes y modos de construcción	41
5.8.2	Visualización mediante Gizmos	42
5.8.3	Telemetría: el cálculo de la memoria estimada	42
5.9	Iteración 8: API pública, optimización del rendimiento	43
5.9.1	API de navegación y consultas espaciales	44
5.9.2	Optimizaciones de rendimiento	44

5.10	Iteración 9: Evasión local entre agentes	45
5.10.1	El algoritmo ORCA	45
5.10.2	Integración con el sistema de <i>jobs</i>	45
5.11	Discusión global del sistema	46
Lista de acrónimos		48
Glosario		49
Bibliografía		51

Índice de figuras

1.1	Ejemplo de malla 2.5D generada por NavMesh sobre un entorno tridimensional.	2
1.2	Visualización de los 6 grados de libertad.	3
2.1	Interfaz del editor de Unity. A la izquierda (en rojo) jerarquía de <code>GameObject</code> y a la derecha (en azul) inspector de componentes de un objeto.	7
2.2	Comparativa de POO y DOD.	9
2.3	Versión 2D de los códigos de Morton.	13
2.4	Ejemplo curva Catmull Rom interpolando los puntos de control.	14
3.1	Visualización de navegación por cualquier superficie.	18
4.1	Diagrama de Gantt de la planificación temporal inicial.	25
4.2	Diagrama de Gantt de la ejecución real del proyecto.	27
5.1	Arquitectura modular del sistema y sus dependencias.	29
5.2	Hoja del SVO y su máscara de ocupación de 64 bits.	30
5.3	Codificación de un código de Morton mediante intercalado de bits.	31
5.4	Disposición de los 32 bits del enlace (SVOLink).	32
5.5	Barrido jerárquico de lo grueso a lo fino durante la rasterización.	33
5.6	Rasterización de hojas en dos pasadas: descarte grueso y muestreo fino.	34
5.7	Pasada gruesa de descarte frente a pasada fina de muestreo.	34
5.8	Enlazado de la vecindad mediante herencia del nodo padre.	35
5.9	Expansión de A* sobre un grafo de resolución heterogénea.	36
5.10	Esquema del bucle principal de la búsqueda A* adaptada.	36
5.11	Recorrido DDA para la comprobación de línea de visión.	38
5.12	Evaluación de un segmento del <i>spline</i> de Catmull-Rom.	39
5.13	Reinterpretación de los bits de un <code>float</code> como un <code>int</code> mediante una unión.	40
5.14	Corrección del <i>clipping</i> mediante el inflado de obstáculos por el radio del agente.	41

5.15	Visualización del SVO en la vista de escena mediante Gizmos.	42
5.16	Estimación de la memoria ocupada por el SVO.	43
5.17	Panel de estadísticas del volumen en el inspector del editor.	43
5.18	Resolución de la velocidad segura en el espacio de velocidades de ORCA. . . .	46
5.19	Flujo de datos global del sistema, del preprocesado a la ejecución.	47

Índice de tablas

3.1	Resumen de las alternativas existentes	20
3.2	Matriz DAFO del proyecto	22
4.1	Presupuesto estimado del proyecto.	26
4.2	Presupuesto final del proyecto tras el seguimiento.	27

Introducción

EL desarrollo de entornos virtuales contemporáneos, como videojuegos y simulaciones interactivas, exige dotar a sus agentes de la inteligencia suficiente para interactuar coherentemente con su entorno. El comportamiento de estas entidades es clave para garantizar la credibilidad, funcionalidad y realismo del entorno virtual, especialmente en escenarios complejos y dinámicos. A pesar de que se trate de un problema que lleva décadas siendo explorado en el campo de la inteligencia artificial [1], a día de hoy, dista de ser trivial cuando se persigue un comportamiento robusto y eficiente.

Dentro de este ámbito, un requisito fundamental es la **navegación autónoma**. Este proceso permite a un agente desplazarse desde un punto de origen hasta un destino específico calculando una ruta viable y esquivando, de manera eficiente, la geometría y los obstáculos presentes en la escena. En el contexto de videojuegos, este comportamiento recae habitualmente en los personajes no jugables, conocidos en inglés como **Non-Playable Character (NPC)**. Su capacidad para desenvolverse de forma autónoma condiciona en gran medida la percepción de inteligencia del sistema. Abordar este reto no solo implica determinar el camino más corto, sino también garantizar que el movimiento resultante sea compatible con las restricciones físicas del entorno y con el comportamiento esperado del agente [2].

El problema de la búsqueda de rutas, conocido en la literatura como *pathfinding*, constituye uno de los casos de estudio más explorados en el campo de la inteligencia artificial aplicada a entornos virtuales [3]. En la actualidad, se considera un problema ampliamente resuelto para agentes terrestres en escenarios estáticos [4], hasta el punto de que los principales motores de desarrollo, tanto comerciales como de código abierto, integran soluciones nativas y robustas para este fin. Ejemplos representativos de ello son el Navigation System en Unreal Engine [5], el sistema nativo de NavMesh en Unity [6] o el componente NavigationServer3D en Godot [7], los cuales permiten implementar comportamientos de navegación complejos con un coste computacional asumible.

La gran mayoría de estas herramientas se fundamentan en la biblioteca de código abierto Recast Navigation [8]. Este sistema permite generar una **mall**a de navegación, que recibe el nombre de NavMesh, y que representa una **aproximación 2.5D** del entorno transitable. Aunque el escenario se modela en tres dimensiones, el cálculo de rutas se proyecta sobre superficies bidimensionales que definen las zonas donde los agentes pueden apoyarse o caminar, tal y como puede apreciarse en la Figura 1.1. A partir de esta geometría, se construye un grafo sobre el cual se aplican algoritmos de búsqueda heurística, siendo el algoritmo A^* el caso estándar de aplicación [9].

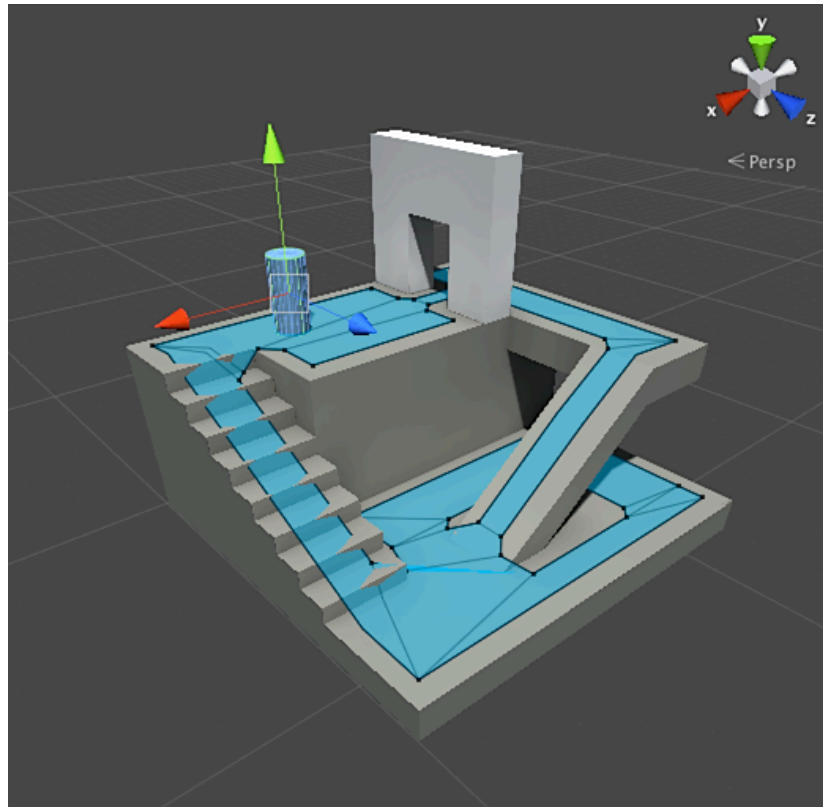


Figura 1.1: Ejemplo de malla 2.5D generada por NavMesh sobre un entorno tridimensional. Imagen extraída de [10].

1.1 Motivación

La necesidad de crear una herramienta que dote a agentes voladores de la capacidad de moverse a través de un espacio tridimensional surge del limitado soporte que ofrecen las tecnologías dominantes en el desarrollo de videojuegos actual. Las soluciones tradicionales basadas en mallas 2.5D resultan ideales para personajes terrestres, ya que presuponen que el agente está anclado al suelo y que su movimiento puede reducirse a un plano horizontal cuya

altura es una propiedad derivada del terreno. Sin embargo, este supuesto no se cumple para entidades con capacidad de vuelo, tales como drones, naves espaciales, enjambres de insectos o criaturas submarinas como peces, las cuales requieren operar con **seis grados de libertad**: tres traslaciones a lo largo de los ejes X , Y y Z y tres rotaciones independientes alrededor de esos mismos ejes, conocidas como *pitch*, *yaw* y *roll*, tal y como se ilustra en la Figura 1.2. Ante la posibilidad de que los agentes puedan modificar su trayectoria en cualquier dirección y altura, un plano transitable resulta insuficiente y se hace imprescindible un **volumen de navegación**. Este debe representar la **totalidad del espacio libre** del entorno y no solo las superficies, logrando garantizar que ninguna ruta viable o alternativa óptima quede excluida durante el cálculo de trayectorias.

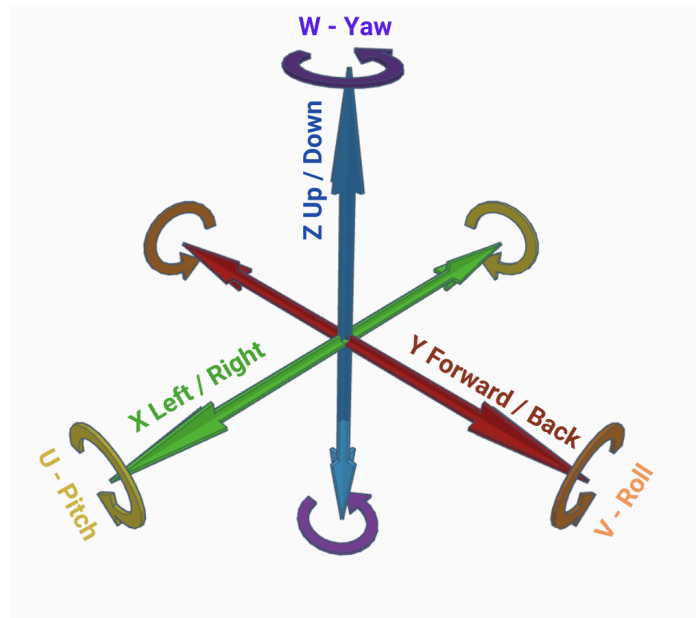


Figura 1.2: Visualización de los 6 grados de libertad. Imagen extraída de [11].

Por consiguiente, el salto hacia una navegación puramente volumétrica plantea un desafío técnico complejo. Al añadir la altura como una dimensión libre, la gran cantidad de espacio a analizar dispara el coste computacional tanto en tiempo como en memoria [12]. Si esta complejidad no se gestiona mediante estructuras de datos espaciales altamente especializadas como *octrees* [13] o representaciones jerárquicas equivalentes, el cálculo de trayectorias provocará un consumo crítico de memoria y una degradación severa en la tasa de *Fotogramas Por Segundo* (FPS), comprometiendo la viabilidad comercial del producto.

Ante la falta de un soporte oficial eficiente en los motores dominantes, los estudios de desarrollo se ven obligados a implementar soluciones *ad hoc* para cada proyecto, o bien a intentar adaptar de forma artificial las tecnologías 2.5D existentes [14]. Ambas alternativas suelen derivar en un incremento significativo de los costes de producción, así como en una

pérdida de tiempo que podría dedicarse al diseño del propio videojuego. Surge, por lo tanto, la necesidad evidente de diseñar un sistema **genérico, optimizado y reutilizable** que resuelva el problema de la navegación tridimensional de manera estructural.

1.2 Objetivos

El objetivo general de este Trabajo de Fin de Grado consiste en el **diseño, implementación y validación en el motor Unity de un sistema de navegación volumétrica eficiente y optimizado**. El resultado final tendrá un enfoque de producto: un paquete de herramientas modular, de código abierto, listo para su integración inmediata en proyectos y entornos de producción reales.

A partir de este objetivo general, se establecen los siguientes objetivos específicos:

- **Diseño arquitectónico:** Diseñar una arquitectura modular y escalable que garantice la eficiencia computacional y facilite la futura integración de nuevas funcionalidades.
- **Representación eficiente del espacio:** Implementar una estructura de datos espacial que represente el volumen navegable de forma compacta y optimizada para consultas en tiempo real.
- **Sistema de serialización:** Desarrollar un módulo de almacenamiento en disco para guardar la representación generada, evitando el coste de su reconstrucción en tiempo de ejecución.
- **Búsqueda de rutas:** Adaptar un algoritmo de búsqueda heurística tridimensional para hallar trayectorias viables dentro del espacio libre del volumen.
- **Postprocesado de trayectorias:** Desarrollar un algoritmo de suavizado de caminos que optimice la trayectoria resultante mediante la eliminación de giros bruscos y nodos redundantes.
- **Integración en Unity:** Desarrollar una herramienta nativa para el editor del motor que simplifique la configuración del sistema y su visualización mediante Gizmos y paneles personalizados.
- **Accesibilidad y usabilidad:** Proveer una interfaz intuitiva que permita la configuración del sistema tanto a perfiles técnicos como a diseñadores de niveles y artistas.
- **Análisis de rendimiento:** Evaluar cuantitativamente la eficiencia global del sistema y de sus componentes críticos bajo diferentes escenarios y cargas de prueba.

- **Distribución modular:** Empaquetar el sistema final bajo los estándares oficiales de Unity para facilitar su integración en otros proyectos y su potencial publicación en la Unity Asset Store.

1.3 Estructura de la memoria

La presente memoria se organiza en los siguientes capítulos:

Capítulo 1: Introducción. El presente capítulo.

Capítulo 2: Fundamentos. Explica los distintos términos y técnicas requeridas para comprender el proyecto.

Capítulo 3: Estado de la cuestión. Examina las soluciones existentes en la literatura y la industria, incluyendo un análisis de [Debilidades, Amenazas, Fortalezas y Oportunidades \(DAFO\)](#).

Capítulo 4: Metodología y planificación. Detalla el modelo de ciclo de vida de software adoptado, así como la planificación temporal y los recursos empleados.

Capítulo 5: Desarrollo. Profundiza en las distintas fases involucradas en la implementación del sistema.

Capítulo 6: Conclusiones. Sintetiza los resultados obtenidos y evalúa las aportaciones del sistema al desarrollo de videojuegos.

Capítulo 7: Trabajos futuros. Propone extensiones y nuevas líneas de investigación que pueden desarrollarse a partir del sistema implementado.

Fundamentos

Para abordar el desarrollo de un sistema de navegación volumétrica, es necesario comprender tanto las tecnologías con las que se va a trabajar como los principios matemáticos o algorítmicos que se usarán en el sistema. En consecuencia, el presente capítulo se estructura en tres bloques. En primer lugar, se tratarán los fundamentos tecnológicos, detallando la arquitectura del motor gráfico seleccionado y las herramientas de alto rendimiento necesarias para garantizar su viabilidad y funcionamiento óptimo. En segundo lugar, se describirán las herramientas y el entorno de desarrollo empleados a lo largo del proyecto, desde el control de versiones hasta la integración continua. Por último, se profundizará en los fundamentos teóricos, los cuales establecerán las bases de las estructuras de datos y las técnicas algorítmicas empleadas en el proyecto.

2.1 Fundamentos tecnológicos

Para crear un sistema robusto y eficiente, es necesario implementarlo dentro de un ecosistema de desarrollo maduro que proporcione las herramientas requeridas. En este contexto, ha sido seleccionado el motor de desarrollo de propósito general Unity [15], introducido en el mercado en el 2005, cuyo uso se justificará en detalle en la Sección 3.2.1 tras un estudio del mercado y alternativas existentes. A continuación se describen sus componentes arquitectónicos y tecnologías más relevantes para el proyecto.

2.1.1 Unity y su modelo de componentes

Unity emplea una arquitectura basada en el patrón *component*, la cual prioriza la composición frente a la herencia jerárquica tradicional de la Programación Orientada a Objetos (POO). En este paradigma, toda entidad presente en el entorno virtual, ya sea un personaje, un elemento decorativo o un elemento de User Interface (UI), se instancia como un objeto genérico de la clase `GameObject`.

Para dotar de comportamiento a estos agentes, se les adjuntan múltiples componentes modulares que heredan de la clase `MonoBehaviour`. Esta modularidad favorece la creación de sistemas muy desacoplados y reutilizables. El ciclo de vida de estos componentes se rige por el patrón *game loop*: tras una fase inicial de configuración mediante los métodos `Awake` y `Start`, el motor ejecuta de manera repetida las funciones `Update`, `LateUpdate` y `FixedUpdate`. El programador debe definir en este ciclo continuo la lógica que se ejecutará en cada frame, como la actualización de la trayectoria del agente volador.

Finalmente todos los `GameObject` se agrupan en una escena de manera jerárquica siguiendo una estructura de árbol (véase la Figura 2.1). Esta constituirá el entorno de ejecución del sistema.

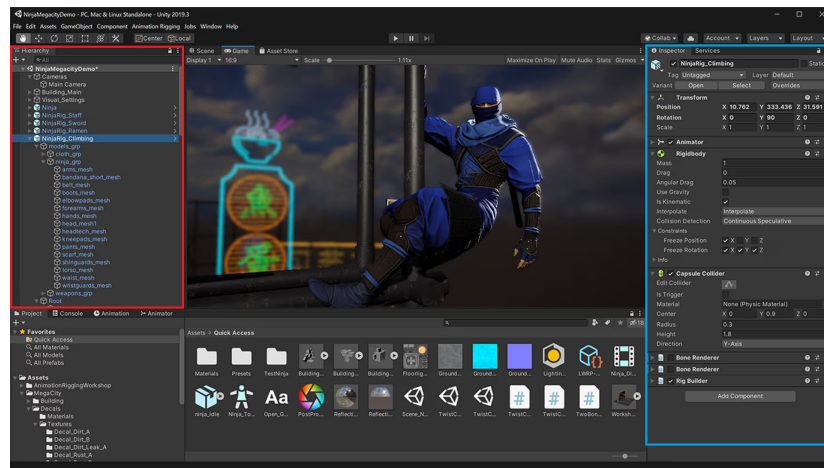


Figura 2.1: Interfaz del editor de Unity. A la izquierda (en rojo) jerarquía de `GameObject` y a la derecha (en azul) inspector de componentes de un objeto. Imagen extraída de [16].

2.1.2 Herramientas proporcionadas por el editor

Un requisito fundamental es facilitar a perfiles no técnicos, como artistas o diseñadores de niveles, la posibilidad de configurar parámetros complejos sin necesidad de alterar el código fuente. Para ello la *Application Programming Interface (API)* de Unity proporciona una serie de herramientas. Mediante el uso de atributos como `[SerializeField]` o `[Range]`, el programador puede exponer en el editor de Unity variables internas de los componentes de modo que estas puedan ser editadas desde fuera. Adicionalmente, heredando de la clase `UnityEditor.Editor`, es posible reescribir por completo la interfaz del sistema, añadiendo paneles personalizados, gráficos o validaciones de datos. A estas capacidades se le suma la clase `Gizmos`, una herramienta de depuración visual que permite renderizar primitivas geométricas, por ejemplo, para proyectar la malla de colisión facilitando así la validación del sistema sin interferir en el producto final.

2.1.3 Motor de físicas y consultas espaciales

El primer paso para que un agente pueda navegar por un entorno tridimensional, es la extracción de información geométrica del escenario. Unity delega la detección de colisiones en su motor de físicas integrado, basado en la biblioteca NVIDIA PhysX [17]. Este sistema utiliza *colliders* (volúmenes matemáticos como cajas, cápsulas o mallas tridimensionales) para representar físicamente a los objetos. Estos pueden ser categorizados en capas con el objetivo de filtrar interacciones entre estas.

La API de Unity proporciona consultas espaciales que permiten consultar la ocupación del entorno. Por ejemplo, la función `OverlapBox` devuelve el conjunto de *colliders* que intersectan un volumen cúbico determinado, utilizando máscaras de bits para restringir esta consulta a las capas de interés. No obstante, en el contexto de la navegación volumétrica, la gran cantidad de consultas requeridas para mapear el espacio aéreo supondría un cuello de botella, agravado por la restricción arquitectónica de Unity de ejecutar la API de físicas de forma síncrona en el hilo principal [18]. Para solventar este impedimento, se hace uso de estructuras como el `OverlapBoxCommand`, la cual permiten agrupar múltiples consultas físicas (*batches*), para su posterior ejecución asíncrona y paralelizada.

2.1.4 Data-Oriented Technology Stack

Históricamente, Unity se ha fundamentado en la POO pura. No obstante, en muchos sistemas que demandan procesamiento masivo de datos, las direcciones de memoria, dispersión de los objetos en la *heap* y el empaquetado ineficiente de datos provocan fallos continuos de caché degradando de manera muy severa el rendimiento.

Para combatir esta limitación, Unity ha desarrollado el ecosistema *Data Oriented Technology Stack* (DOTS) [19]. Este paradigma, basado en el *Diseño Orientado a Datos* (DOD), busca maximizar el rendimiento estructurando los datos en memoria de forma que la *Central Processing Unit* (CPU) pueda procesar la información de manera óptima aprovechando la arquitectura del hardware (ilustrado en la Figura 2.2).

Los componentes clave de este *stack* tecnológico son los siguientes:

- **C# Job System:** API que abstrae y simplifica la programación concurrente, permitiendo y simplificando la ejecución segura de código multihilo. Funciona mediante unidades de cómputo que reciben el nombre de *jobs*, las cuales declaran explícitamente sus dependencias de datos [21]. Esta arquitectura previene errores en tiempo de ejecución como es el caso de las *race conditions*.
- **Burst Compiler:** compilador avanzado diseñado para traducir el lenguaje C# directamente a código máquina nativo, eludiendo así la máquina virtual estándar de este len-

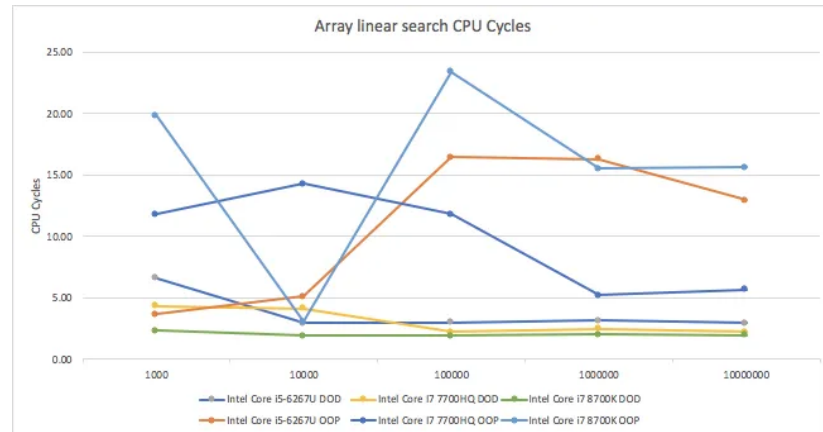


Figura 2.2: Comparativa de POO y DOD. Imagen extraída de [20].

guaje conocida como **Common Language Runtime (CLR)** [22]. Basado en la infraestructura **LLVM** [23], optimiza el código aprovechando las instrucciones **Single Instruction, Multiple Data (SIMD)** que permiten que el procesador ejecute una misma operación matemática de manera simultánea sobre un conjunto de datos, acelerando drásticamente los cálculos.

- **Unity.Mathematics:** biblioteca matemática diseñada para aprovechar al máximo el Burst Compiler [24]. Proporciona una serie de tipos de datos vectoriales como el *float3* o *int3*.
- **Colecciones nativas:** Estructuras de datos (tales como el `NativeArray`, `NativeList` o `NativeParallelMultiHashMap`) que reemplazan las colecciones estándar de C#. Estas operan sobre memoria no administrada lo que permite suprimir picos de latencia provocados por el recolector de basura. Al usar estas colecciones, el programador asume el control absoluto y la responsabilidad de reservar y liberar explícitamente la memoria [25].

2.1.5 Características de bajo nivel en C#

Para complementar el ecosistema de **DOTS** y asegurar un rendimiento óptimo, resulta indispensable emplear características de bajo nivel del lenguaje. Las siguientes técnicas minimizan el impacto del recolector de memoria y reducen costes de transferencia de datos en zonas de ejecución críticas:

- **ref readonly:** permite devolver una referencia de solo lectura a una estructura de datos. Al evitar la copia de estructuras complejas, se reduce notablemente el uso de me-

moria. Adicionalmente, la palabra clave `readonly` garantiza a nivel de compilación la integridad de los datos originales.

- **Span<T>**: representa una sección de memoria contigua independientemente de si los datos residen en el `stack` o en la `heap`. Esta estructura permite realizar operaciones de lectura y escritura sobre secciones de esta sin necesidad de almacenar más memoria.
- **stackalloc**: permite reservar memoria en el `stack`. De este modo evita al recolector de basura tener que limpiar colecciones que se usan a modo de *buffers* temporales, asegurando su liberación inmediata al salir de la función y omitiendo por completo al recolector de basura.

A este conjunto de técnicas se añade la manipulación de bits. Esta técnica permite codificar varios campos en una única variable entera, reduciendo drásticamente el uso de memoria.

Estas técnicas de bajo nivel se aplican de forma recurrente a lo largo del Capítulo 5. A modo de ejemplo, la manipulación de bits resulta esencial en la codificación compacta de las hojas y los punteros de la estructura de datos espacial (Sección 5.2). El acceso por referencia de solo lectura (`ref readonly`) se emplea para optimizar las rutas más críticas del buscador (Sección 5.9). Por último, la reserva de memoria en el `stack` mediante `stackalloc`, junto con su manipulación a través de estructuras `Span<T>`, se aprovecha en el solucionador de evasión local, que se ejecuta en cada frame del sistema por lo que se debe evitar generar presión sobre el recolector de basura (Sección 5.10).

2.2 Herramientas y entorno de desarrollo

A parte de las tecnologías que se emplean en el sistema, su desarrollo requirió un conjunto de herramientas que conformasen un entorno de trabajo productivo. A continuación se recogen todas aquellas que fueron empleadas a lo largo del proyecto:

- **Entorno de programación**: la creación de escenas, la configuración de los componentes y la depuración visual se realizan sobre el editor del motor Unity [15], mientras que la escritura del código fuente en C# se apoya en el entorno de desarrollo integrado Visual Studio [26], que ofrece asistencia a la edición, navegación por el código y depuración.
- **Control de versiones**: la evolución del código se gestiona con el sistema de control de versiones distribuido Git [27], empleando la plataforma GitHub [28] como servicio de alojamiento remoto del repositorio y soporte para la revisión e integración de los cambios.

- **Integración continua:** la verificación automática del proyecto se delega en un sistema de *Continuous Integration* (CI) implementado mediante GitHub Actions [29], que ejecuta de forma desatendida la batería de pruebas en una máquina remota ante cada propuesta o integración de cambios.
- **Calidad y estilo del código:** la consistencia del formato se garantiza con el formateador CSharpier [30], que aplica de manera automática una guía de estilo común a todo el código fuente.
- **Pruebas automatizadas:** la validación del comportamiento se construye sobre el Unity Test Framework [31], que permite definir pruebas tanto en modo de edición (*EditMode*) como en modo de ejecución (*PlayMode*).

2.3 Fundamentos teóricos

Para el diseño de una herramienta de navegación tridimensional robusta, la elección de las tecnologías de bajo nivel constituye únicamente la mitad del problema. La otra mitad reside en la selección e implementación de los algoritmos y estructuras de datos adecuados que dictaminarán la eficiencia y rendimiento del sistema. A continuación, se describen los principios teóricos que constituyen el campo de la navegación volumétrica.

2.3.1 Búsqueda de caminos

La búsqueda de caminos, o *pathfinding*, consiste en el problema computacional de determinar una ruta continua y libre de colisiones que conecte un punto de origen con un destino dentro de un entorno virtual [3]. Dado que el espacio físico es continuo y contiene infinitas posiciones posibles, el primer paso para su resolución exige obtener una representación discreta y estructurada de la geometría de la escena. Esta abstracción permitirá aplicar, posteriormente, un algoritmo de búsqueda de caminos mínimos.

Representación del espacio navegable

Al estudiar el ámbito de la navegación volumétrica, se puede observar que existen tres alternativas predominantes. Estas se clasifican en:

- **Waypoints:** Esta solución requiere el posicionamiento manual de una serie de nodos que reciben dicho nombre. Estos puntos constituirán los vértices del grafo y deben ubicarse por completo dentro del espacio libre de colisiones. Las aristas se corresponderán a aquellas conexiones de vértices para las cuales existe una línea de visión directa, es decir, sin obstáculos intermedios entre ellos.

A pesar de ser computacionalmente el método más ligero, este carece de escalabilidad en escenarios de gran tamaño o en entornos dinámicos, ya que cualquier cambio en la geometría de la escena obliga a recolocar los nodos y recalcular sus conexiones. Sin embargo, dado que la implementación de un sistema con estas características es trivial, es común encontrar estudios que recurren a soluciones *ad hoc* cuando no se requieren sistemas de navegación complejos [3].

- **Campos de flujo (*Flow fields*):** Esta aproximación consiste en calcular un campo vectorial global sobre una cuadrícula o volumen espacial, donde cada celda almacena un vector de dirección que apunta hacia el destino. Si bien resultan extremadamente eficientes para movilizar enjambres o multitudes masivas hacia un objetivo común, su coste computacional y uso de memoria en entornos tridimensionales se vuelve prohibitivo cuando existen múltiples agentes persiguiendo destinos independientes, descartándolos para soluciones de propósito general [32].
- ***Octrees*:** Constituyen la técnica más extendida en la industria. En ella, el espacio se subdivide en octantes, creando una estructura de datos jerárquica que representa el espacio libre navegable [13]. En prácticamente la totalidad de los casos, estas representaciones se optimizan haciendo uso de una modificación estructural: los *Sparse Voxel Octree (SVO)*. Estos dividen solo aquellos octantes que contienen geometría, logrando así un ahorro de memoria considerable.

La discretización del volumen mediante estas estructuras requiere localizar rápidamente las celdas o *vóxeles* en la memoria. Para lograrlo, resulta fundamental mapear coordenadas tridimensionales (X, Y, Z) a un índice escalar, preservando la localidad espacial. Esto se consigue mediante el cálculo de códigos de Morton, basados en las curvas de orden Z [33] (véase la Figura 2.3). A través de operaciones a nivel de bits, se intercalan los bits representativos de cada coordenada espacial para formar un único número entero. Esta técnica garantiza que nodos espacialmente adyacentes en la escena tiendan a ocupar posiciones contiguas en la memoria lineal, minimizando los fallos de caché y maximizando los beneficios arquitectónicos del paradigma *DOD* ya explicados en la Sección 2.1.4.

Algoritmos de búsqueda en grafos

Una vez estructurado el espacio en un grafo navegable, la extracción de la trayectoria se delega en algoritmos de caminos mínimos. El algoritmo clásico de Dijkstra explora los nodos de forma uniforme garantizando siempre la ruta más corta [35]. No obstante, su expansión radial conlleva un coste computacional inasumible para cálculos en tiempo real dentro de grandes volúmenes.

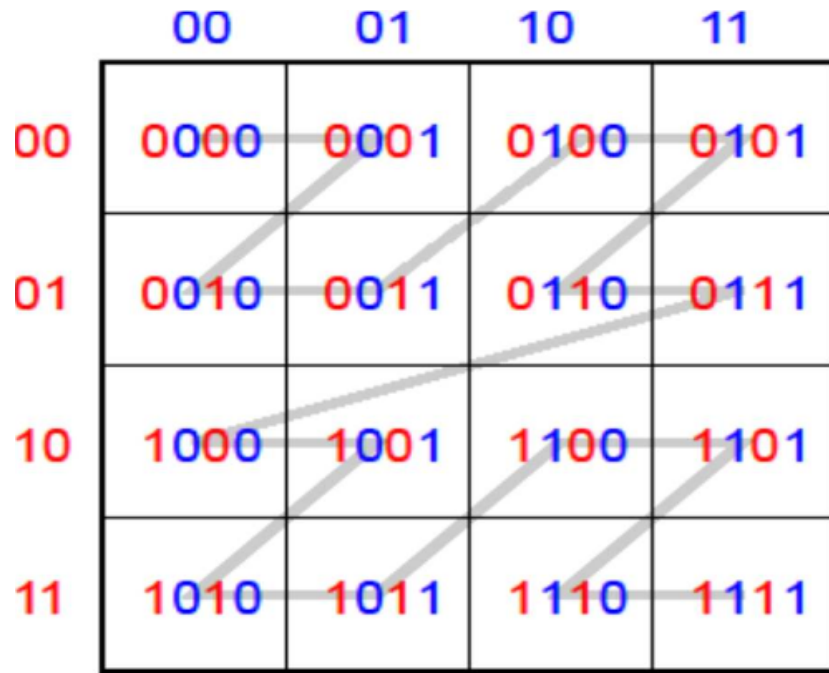


Figura 2.3: Versión 2D de los códigos de Morton. Imagen extraída de [34].

En respuesta a esta limitación, el estándar de la industria es el algoritmo A^* . Esta técnica optimiza la búsqueda introduciendo una heurística a la función de coste. Dicha función se define matemáticamente como $f(n) = g(n) + h(n)$, donde $g(n)$ es el coste real desde el origen y $h(n)$ es el coste estimado hasta la meta. Esta estimación permite priorizar los nodos más prometedores, orientando la búsqueda hacia el destino y reduciendo exponencialmente el número de celdas evaluadas [9].

Postprocesado de trayectorias

Las rutas generadas por algoritmos sobre volúmenes discretizados resultan muy artificiales al estar compuestas por multitud de segmentos. Para dotar al agente de un movimiento orgánico, el sistema requiere de una etapa de postprocesado que se suele dividir en dos fases.

En primer lugar, se aplica un algoritmo de *String Pulling* [36] con su variante voraz. Este itera sobre la ruta buscando el nodo más lejano con el que exista línea de visión directa, descartando recursivamente todos los nodos intermedios redundantes. Para validar esta línea de visión se requiere trazar rayos y comprobar su intersección con la geometría. Esto requiere que el algoritmo no sea solo correcto sino que también responda de manera robusta a errores derivados de la falta de precisión decimal, siendo el algoritmo *Digital Differential Analyzer* (DDA) [37] el caso estándar de aplicación.

En segundo lugar, la trayectoria depurada se suaviza mediante la interpolación con curvas paramétricas. Se suele optar por el *spline* de Catmull-Rom (como se muestra en la Figura 2.4) sobre alternativas como las curvas de Bézier o B-splines. Esta decisión se fundamenta en que este tipo de curvas pasa de forma exacta a través de todos los puntos de control suministrados [38].

Como se ilustra en la figura, los puntos rojos (denotados con la letra P) representan estos **puntos de control** o nodos de la trayectoria original. Dado que la etapa previa asegura que estos nodos están ubicados en espacio libre, obligar a la curva a cruzar exactamente por ellos, garantizando que la trayectoria resultante no intersecte accidentalmente con la geometría del entorno.

Por su parte, la curva negra y las etiquetas Q representan los **segmentos** generados matemáticamente que unen dichos puntos de control. El trazado de estos segmentos se define mediante el parámetro de interpolación u (ilustrado con marcas amarillas), el cual toma valores entre 0 y 1 para calcular las posiciones intermedias a lo largo del segmento, permitiendo así una transición suave entre un punto de control y el siguiente.

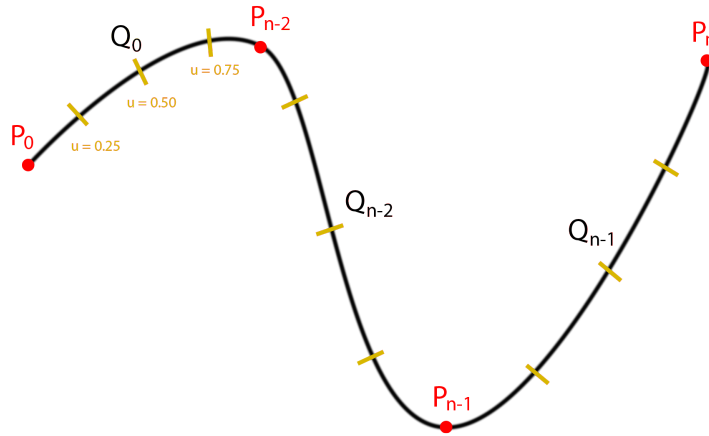


Figura 2.4: Ejemplo curva Catmull Rom interpolando los puntos de control. Imagen extraída de [39].

2.3.2 Planificación de caminos

A pesar de haber encontrado un camino libre de obstáculos, en múltiples ocasiones se requiere la replanificación de este, por ejemplo, para esquivar a otros agentes. A este proceso se le conoce como *path planning*, siendo el algoritmo [Optimal Reciprocal Collision Avoidance](#)

(ORCA) uno de los más relevantes [40]. Este algoritmo se basa en los llamados obstáculos de velocidad, correspondiéndose estos con los propios agentes dinámicos de la escena. Para lograr la evasión, ORCA razona sobre el denominado *espacio de velocidades*: un dominio continuo en el que cada punto representa un posible vector de velocidad que el agente podría adoptar en el instante actual. Dentro de este dominio, el algoritmo delimita la región de velocidades **prohibidas** (aquellas que, de mantenerse, conducirían a una colisión) frente a las velocidades **permitidas**, consideradas seguras. La particularidad del método reside en su premisa de reciprocidad: ambos agentes ejecutan ORCA de forma simultánea y asumen de manera conjunta la responsabilidad de esquivarse, en lugar de delegar toda la maniobra en uno solo de ellos.

A modo de ejemplo ilustrativo, considérense dos agentes voladores, A y B , que se aproximan frontalmente a lo largo de una misma recta. La velocidad preferida de A , aquella que lo dirige directamente hacia su destino, apunta hacia B , por lo que recae dentro de su región de velocidades prohibidas: mantenerla garantizaría el impacto. El algoritmo calcula entonces el mínimo vector de corrección $\vec{u}$ que situaría la velocidad relativa de ambos justo sobre la frontera de la región segura, por ejemplo, desviándose ligeramente hacia un lado. Si B fuese un obstáculo estático incapaz de reaccionar, A tendría que asumir esa corrección en su totalidad. No obstante, dado que B también ejecuta el algoritmo, la responsabilidad se reparte de forma equitativa: A modifica su velocidad en $\vec{u}/2$ y B aplica la otra mitad en sentido opuesto. Así, ambos describen trayectorias simétricas y la colisión se evita con la mitad del esfuerzo por cada parte, lo que elimina además las oscilaciones que surgirían si cada agente supusiese que el otro no va a alterar su rumbo.

En términos operacionales, ORCA modela matemáticamente estas restricciones definiendo un semiplano de velocidades seguras para cada par de agentes en conflicto. A partir de la posición relativa, los radios de colisión y las velocidades actuales, se computa la mínima alteración de velocidad necesaria para garantizar que las trayectorias no se intersequen. Gracias a la premisa de reciprocidad, el algoritmo asume que cada entidad aplicará exactamente la mitad de esta corrección. Para ello, el sistema emplea técnicas de programación lineal para calcular de manera eficiente una nueva velocidad óptima que satisfaga todas las restricciones geométricas de los semiplanos y, a la vez, se aproxime lo máximo posible a la velocidad preferida del agente (aquella que lo dirige hacia su meta).

El principal problema de este sistema de evasión local es que los agentes se pueden quedar atascados ante obstáculos dinámicos muy grandes. Resolver este problema, denominado **deadlock**, conlleva hacer uso de sistemas de planificación que trabajen a más alto nivel y que operen independientemente del sistema de navegación, lo cual escapa del alcance de este proyecto.

2.3.3 Integridad estructural mediante algoritmos Hash

Para evitar reconstruir la representación del espacio navegable cada vez que se carga el juego, los sistemas de navegación suelen proveer un proceso denominado como *bake*. Este consiste en construir las estructuras de datos de manera anticipada desde el editor y guardarlas en disco para que, posteriormente, puedan ser cargadas en tiempo de ejecución.

Para asegurar la integridad de la escena, es decir, garantizar que no se han producido cambios en su geometría desde la última vez que se realizó el *bake*, se suele computar el *hash* de los objetos correspondientes. De este modo, si el valor resultante varía, se detecta que se ha producido una modificación. En el ecosistema de C#, no se recomienda utilizar el método por defecto `GetHashCode` [41]. Esto se debe a que proporciona resultados dependientes de la semilla de ejecución, por lo que los valores pueden variar entre diferentes sesiones, rompiendo por completo el determinismo.

En su lugar, se suelen implementar algoritmos de *hash* desde cero, como es el caso de *FNV-1a*, el cual destaca por su gran velocidad [42]. Este algoritmo itera directamente sobre los bytes de los datos, acumulando el resultado en un valor inicializado con el llamado *FNV offset basis*. En cada iteración, aplica una operación XOR con el byte correspondiente y, acto seguido, lo multiplica por un número primo conocido como *FNV prime*.

Estado de la cuestión

El presente capítulo ofrece una revisión detallada del panorama actual respecto a las soluciones de navegación espacial existentes en el ámbito de los videojuegos. A lo largo de las siguientes secciones se explorarán las herramientas con mayor relevancia en el mercado actual. Posteriormente, a partir de un análisis crítico de estas, se justificará la necesidad de la herramienta de código abierto propuesta en este trabajo.

3.1 Herramientas disponibles

En la actualidad, existe un amplio repertorio de herramientas orientadas a resolver el problema de la navegación en entornos virtuales. Entre estas soluciones, aquellas que gozan de mayor relevancia y adopción en la industria, se fundamentan en enfoques que combinan estructuras *SVO* y el algoritmo de búsqueda heurística *A** [3], conceptos ya explicados previamente en la Sección 2.3.1. A nivel arquitectónico, estas soluciones se dividen en dos grandes grupos: aquellas que operan de forma independiente al motor gráfico y las diseñadas para integrarse en un entorno de desarrollo concreto.

3.1.1 Herramientas genéricas

Si se analizan el conjunto de herramientas que son agnósticas al motor, se observa que la solución predominante en el mercado es la de Mercuna [43]. Esta herramienta fue concebida originalmente como un *plugin* comercial exclusivo para Unreal Engine [44]. Sin embargo, su éxito motivó su posterior extensión para operar de forma desacoplada a este.

El principal atractivo de Mercuna radica en su capacidad para unificar, bajo un mismo ecosistema, casi todos los casos de uso que engloba la navegación. Entre sus características más relevantes se encuentran:

- **Navegación volumétrica:** Permite a los agentes tanto voladores como submarinos, moverse por la totalidad del espacio navegable, haciendo uso completo de sus seis grados de libertad.
- **Navegación sobre superficies:** Proporciona la capacidad de desplazarse sobre cualquier tipo de geometría. Esto no se limita al suelo convencional, sino que incluye también paredes o techos, gestionando la gravedad de manera local al agente (ver Figura 3.1).
- **Soporte para agentes no holonómicos:** Incorpora las restricciones físicas y cinemáticas de los agentes en la fase de cálculo de trayectorias. De este modo, es posible modelar el comportamiento de vehículos con radios de giro limitados o entidades que carecen de la capacidad de acelerar y frenar de manera instantánea.

Se trata, sin lugar a dudas, de la herramienta que se puede considerar más robusta y completa del mercado. Sin embargo, su modelo de negocio supone una barrera de entrada prohibitiva para la mayor parte de estudios. Mercuna dispone de distintas licencias que dependerán del subconjunto de características requerido, con costes que parten de £15000 o de £2500 para estudios con un presupuesto de desarrollo inferior a \$500000 [45].

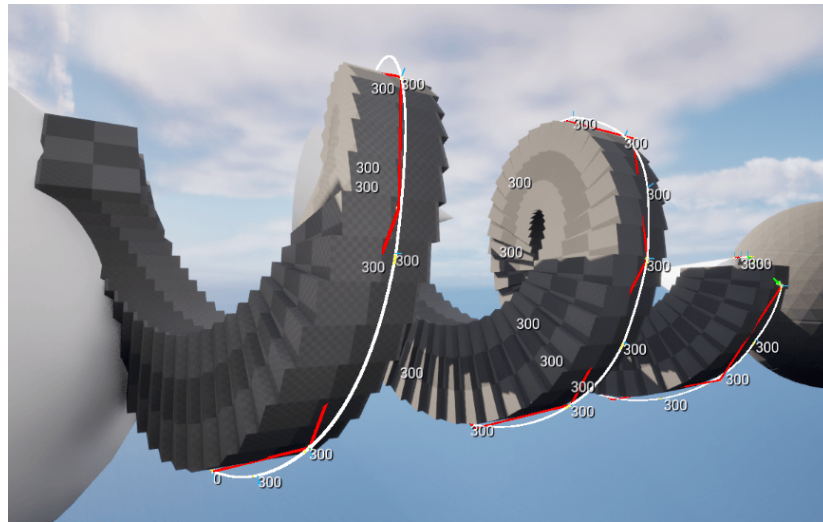


Figura 3.1: Visualización de navegación por cualquier superficie. Imagen extraída de [46].

3.1.2 Soluciones específicas de motor

Por el contrario, las herramientas especializadas en un motor gráfico concreto, operan en un paradigma de desarrollo diferente. Al prescindir de una arquitectura genérica y agnóstica que asegure su interoperabilidad multiplataforma, estas soluciones pueden aprovechar las

rutinas nativas del motor para optimizar el rendimiento computacional, ofreciendo asimismo una integración total con el editor. Esto se traduce en herramientas mucho más intuitivas para el desarrollador final.

En este ámbito predomina el ecosistema de Unity, respaldado por su tienda oficial de recursos: la Asset Store [47]. Esta plataforma permite comercializar tanto herramientas de desarrollo como recursos artísticos (modelos tridimensionales, texturas o composiciones musicales, entre otros) que aceleran los flujos de trabajo al evitar la necesidad de diseñar dichos componentes desde el principio. Bajo un modelo de reparto de ingresos, Unity proporciona visibilidad de los productos creados.

Como consecuencia de la vasta magnitud de este ecosistema, casi todas las herramientas de terceros se desarrollan de manera exclusiva para este motor. Mientras que en entornos como Unreal Engine las soluciones complejas suelen comercializarse con costosas licencias comerciales, la Asset Store de Unity proporciona productos más asequibles para estudios independientes o de menor presupuesto. Entre estas destacan:

Nav3D (Softbrix Studio)

Desarrollada por Softbrix Studio, Nav3D es la opción mejor valorada en la tienda de Unity (Asset Store) [48]. Su mayor virtud reside en que no se limita al cálculo de rutas estáticas, sino que integra un sistema de evasión local. Esto garantiza que los NPC, en escenarios multiagente, coordinen sus trayectorias y las reajusten en tiempo real para evitar colisionar entre ellos [49].

Nav3D (m6io)

Creada por el desarrollador independiente m6io, esta solución se comercializa con un enfoque estrictamente técnico. A pesar de cubrir la mayoría de casos de uso, carece por completo de herramientas visuales e interfaces de configuración del editor. En consecuencia, presenta una curva de aprendizaje mucho más pronunciada, exigiendo al usuario un nivel avanzado de programación en el lenguaje C# [50].

HyperNav

Elaborada por Infohazard Games, constituye actualmente una de las alternativas más robustas y sofisticadas del mercado de Unity, lo cual se refleja también en un coste superior. Su principal ventaja competitiva es su enfoque multiparadigma: además de la navegación puramente volumétrica, ofrece navegación sobre superficies, permitiendo a los agentes desplazarse por suelos y paredes con una gestión automatizada de la gravedad local [51].

Spatial Pathfinding

Desarrollada por TecMaid, se presenta como una alternativa más económica para desarrolladores con presupuestos ajustados. Se trata de una solución que sacrifica funcionalidades avanzadas en favor de ofrecer un sistema más ligero, directo y de implementación rápida [52].

3.2 Comparativa de mercado

A modo de síntesis, la herramienta de carácter agnóstico Mercuna ofrece una solución tecnológicamente superior, si bien su modelo comercial la orienta exclusivamente a producciones de alto presupuesto conocidas como AAA. Por otro lado, las alternativas nativas para Unity democratizan el acceso a estas tecnologías con precios mucho más asequibles, situándose por lo general por debajo de los 100 euros.

Sin embargo, estas últimas opciones se distribuyen bajo modelos de **código cerrado**, lo que supone una desventaja técnica importante. Esta opacidad impide a los desarrolladores consultar el código y prever el impacto real de cada funcionalidad en el rendimiento, lo que habitualmente deriva en un uso subóptimo del sistema. Además, en muchos casos, estas soluciones carecen de un nivel de refinamiento adecuado en cuanto a usabilidad; como es el caso del ya mencionado anteriormente Nav3D de m6io, que prescinde por completo de herramientas gráficas. Esta comparativa general queda reflejada de manera resumida en la Tabla 3.1.

Herramienta	Plataforma	Enfoque	Precio
Mercuna	Multiplataforma	Multiparadigma	£15000 +
Nav3D (Softbrix Studio)	Unity	Volumétrica	64,39€
Nav3D (m6io)	Unity	Volumétrica	59,80€
HyperNav	Unity	Volumétrica + Superficies	82,80€
Spatial Pathfinding	Unity	Volumétrica	27,59€

Tabla 3.1: Resumen de las alternativas existentes

3.2.1 Justificación del entorno de desarrollo

En base a las soluciones analizadas, la elección de Unity como entorno para de desarrollo resulta óptima para este trabajo. El ecosistema de la Asset Store garantiza no solo que la solución responda a una demanda real, sino que también asegura un canal con la visibilidad suficiente para su correcta difusión.

A este factor se suma una ventaja técnica fundamental: la disponibilidad de herramientas orientadas al alto rendimiento integradas de forma nativa en el motor, como es el caso del Burst Compiler. Esta tecnología permite compilar código C# en código máquina muy optimizado mediante el uso de la infraestructura LLVM [23]. Dado que la navegación volumétrica implica una carga computacional intensiva, aprovechar el Burst Compiler facilita la ejecución de estas rutinas matemáticas en *hilos secundarios* con un impacto mínimo en el *hilo principal de procesamiento*, preservando así la tasa de refresco del entorno interactivo.

3.3 DAFO

Con el propósito de evaluar la viabilidad estratégica y justificar el desarrollo del proyecto, se ha llevado a cabo un análisis DAFO. Este estudio se estructura en un bloque de **análisis interno** (orientado a identificar las fortalezas y debilidades inherentes a la propuesta), un bloque de **análisis externo** (centrado en identificar las oportunidades y amenazas que presenta el mercado actual) y un apartado de conclusiones.

3.3.1 Análisis interno

La principal fortaleza del sistema propuesto radica en su condición de ser la primera herramienta de código abierto específicamente orientada a la navegación volumétrica. A diferencia de las alternativas comerciales cerradas, este enfoque permite a los usuarios auditar el sistema, adaptarlo mediante modificaciones personalizadas según las necesidades de su proyecto y, potencialmente, contribuir a la evolución de este. Asimismo, la transparencia en el diseño del algoritmo permite a los desarrolladores prever con precisión el impacto en el rendimiento de cada componente.

Por el contrario, la principal debilidad se asocia a su naturaleza como proyecto emergente dentro de un mercado maduro y consolidado por soluciones veteranas. Esto puede generar una percepción inicial de falta de madurez o robustez, lo que exigirá un proceso continuo de iteración y refinamiento técnico basado en la retroalimentación recibida tras las fases iniciales de despliegue.

3.3.2 Análisis externo

Las oportunidades del proyecto emergen ante la carencia de soluciones integradas o gratuitas de navegación volumétrica en las versiones nativas de los motores gráficos actuales. En este contexto, la existencia de un mercado centralizado como la Asset Store, permitirá dar a conocer la herramienta directamente a desarrolladores independientes.

Por último, las amenazas provienen del dinamismo y la constante evolución tecnológica que caracterizan al sector de los videojuegos. El auge de motores alternativos de código

abierto, como Godot [53], los cuales incorporan continuamente nuevas capacidades [54], representa un factor de competitividad externa. Asimismo, la eventual inclusión de una herramienta de navegación tridimensional oficial y nativa por parte de la propia compañía Unity constituye el riesgo más significativo para soluciones desarrolladas por terceros.

Fortalezas	Debilidades
• Primera herramienta de código abierto en su campo. • Transparencia total, permitiendo optimización y adaptabilidad por parte del usuario.	• Proyecto emergente en un mercado con alternativas consolidadas. • Necesidad inicial de cambios para adaptarse a los requisitos de la comunidad.
Oportunidades	Amenazas
• Carencia de herramientas integradas gratuitas de navegación volumétrica. • Gran canal de difusión directa a través de la Asset Store.	• Sector de los videojuegos en constante cambio y evolución. • Crecimiento de motores emergentes competitivos (ej. Godot). • Riesgo de aparición de una herramienta oficial integrada en el motor.

Tabla 3.2: Matriz DAFO del proyecto

3.3.3 Conclusiones

A partir de la matriz DAFO expuesta en la Tabla 3.2, se concluye que el desarrollo de este sistema es altamente viable. A pesar de que el proyecto deberá competir con soluciones comerciales maduras, su naturaleza de código abierto representa una gran ventaja competitiva. En conclusión, el proyecto se posiciona como una alternativa democratizadora, accesible y con un claro potencial de adopción dentro del vasto ecosistema de desarrolladores de Unity.

Metodología y planificación

El presente capítulo detalla los aspectos relativos a la gestión del proyecto. Para ello, primero se define la metodología de desarrollo software adoptada, detallando tanto el modelo de ciclo de vida empleado como las prácticas de ingeniería que garantizan la calidad y estabilidad del producto. Posteriormente, se presenta la planificación inicial con la estimación de tiempos y costes asociados al desarrollo. Por último, se lleva a cabo un seguimiento crítico de dicha planificación, contrastando las previsiones iniciales con la ejecución real del trabajo.

4.1 Metodología de desarrollo

Para el desarrollo del proyecto se ha adoptado una metodología iterativa e incremental [55]. Se estima que este enfoque se ajusta de manera óptima para gestionar la complejidad de un sistema de navegación volumétrica, ya que permite dividir el problema global en incrementos funcionales más manejables. De este modo, en lugar de afrontar la construcción del sistema como un bloque monolítico, el producto avanza por fases. Cada iteración engloba tareas de análisis de requisitos, diseño técnico, implementación y pruebas. Esta estructura garantiza que cada nuevo componente sea robusto antes de integrarse en el conjunto, lo que facilita la detección temprana de errores y la validación continua tras cada ciclo.

El punto de partida es un producto mínimo viable centrado en la viabilidad técnica: la construcción de la estructura de datos espacial en tiempo de ejecución y una primera versión de la búsqueda de rutas. A partir de este núcleo, cada iteración incorpora una nueva funcionalidad o mejora, evaluando siempre la corrección del comportamiento y su impacto en el rendimiento global. Para facilitar esta validación, el código se divide en módulos con responsabilidades bien delimitadas mediante ensamblados, lo que permite excluir el código de edición de las compilaciones finales.

4.2 Planificación inicial

La estimación inicial de tiempo se realizó conforme a la normativa de los créditos [European Credit Transfer and Accumulation System \(ECTS\)](#). Esto implica unas **360** horas de trabajo, distribuidas a lo largo del segundo cuatrimestre del curso, entre los meses de marzo y junio de 2026, en un período aproximado de catorce semanas.

4.2.1 Planificación temporal

Siguiendo la metodología iterativa, el trabajo se organizó en las siguientes fases:

1. **Estudio previo y análisis del estado de la cuestión:** revisión bibliográfica de las técnicas de navegación, análisis de soluciones comerciales y formación en las tecnologías de alto rendimiento del motor.
2. **Diseño de la arquitectura:** definición de la estructura modular y selección de algoritmos y estructuras de datos.
3. **Implementación de la estructura de datos volumétrica:** construcción del espacio navegable mediante un [SVO](#), incluyendo su rasterización y serialización en disco.
4. **Implementación de la búsqueda y postprocesado de rutas:** adaptación del algoritmo de búsqueda heurística tridimensional y desarrollo de la etapa de suavizado de trayectorias.
5. **Integración en Unity:** desarrollo de los componentes, las herramientas de editor y los indicadores visuales [GIZMOS](#) que dan acceso al sistema al usuario final.
6. **Evaluación de rendimiento y optimización:** diseño de escenas de prueba representativas y medición de los tiempos de construcción y búsqueda, así como del consumo de memoria, orientando las sucesivas optimizaciones.
7. **Redacción de la memoria:** documentación del diseño, la implementación, los resultados y las conclusiones del proyecto.

La distribución temporal de estas fases se recoge en el diagrama de Gantt de la Figura 4.1. Es importante señalar el solapamiento entre varias de ellas, consecuencia de que el diseño puede comenzar antes de haber solventado por completo los errores de las etapas previas. Tanto las tareas de validación y pruebas como la propia redacción de la memoria se prolongan a lo largo de prácticamente todo el proyecto.

Los hitos marcados se corresponden con: **H1** (semana 3), cierre del estudio del estado de la cuestión; **H2** (semana 7), finalización de la estructura de datos volumétrica; **H3** (semana 10),

Fase	Marzo		Abril				Mayo				Junio			
Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Estado del arte y estudio previo														
Diseño de la arquitectura														
Estructura de datos volumétrica														
Búsqueda y postprocesado de rutas														
Integración en Unity														
Evaluación y optimización														
Validación y pruebas (CI)														
Redacción de la memoria														
Hitos			◆				◆			◆			◆	◆

Figura 4.1: Diagrama de Gantt de la planificación temporal inicial. Cada columna representa una semana de trabajo. Las barras rosadas corresponden a las fases principales y las azuladas a las tareas transversales. Los rombos (◆) señalan los hitos.

primer cálculo de rutas funcional, que constituye el producto mínimo viable; **H4** (semana 13), integración en Unity y optimización completadas; y **H5** (semana 14), entrega de la memoria.

4.2.2 Estimación de costes

El presupuesto del proyecto tiene en cuenta costes de personal, equipamiento, software y gastos indirectos, siendo el coste personal la parte principal. Partiendo del hecho de que el salario medio en España para un programador *junior* es de 21.955 € [56], se estima que el total correspondiente a las 360 horas es de 4.114,80 €.

El equipamiento requiere un ordenador de gama media valorado en unos 1.200 €. Aplicando una amortización lineal a cuatro años por los tres meses y medio de uso, su coste es de 87,50 €. Respecto al software, todas las herramientas (Unity Personal, Visual Studio Community, Git y GitHub) son gratuitas para este caso de uso, por lo que no generan gastos. Finalmente, se calculan unos costes indirectos (electricidad, internet) del 5 % sobre los costes directos. El presupuesto total, resumido en la Tabla 4.1, asciende a 4.412,42 €.

4.3 Seguimiento

Aunque la planificación inicial se cumplió en gran medida, surgieron algunas desviaciones que alargaron el desarrollo debido a la naturaleza del proyecto. Sin embargo, la metodología iterativa fue clave para solucionar estos imprevistos sin comprometer la estabilidad del sistema.

Concepto	Coste
Recursos humanos (360 h $\times$ 11,43 €/h)	4.114,80 €
Amortización del equipo de desarrollo	87,50 €
Licencias de software	0,00 €
Costes indirectos (5 %)	210,12 €
Total	4.412,42 €

Tabla 4.1: Presupuesto estimado del proyecto.

La desviación más significativa se localizó en la fase de búsqueda de rutas. Si bien la primera versión del algoritmo se completó a tiempo, garantizar su robustez requirió más esfuerzo del estimado. En concreto, las trayectorias generadas atravesaban en determinadas situaciones la geometría del entorno, un fenómeno conocido como *clipping*. La corrección de este comportamiento exigió varias iteraciones sucesivas y, sobre todo, el desarrollo de una batería de pruebas considerablemente más extensa de la prevista, incluyendo *tests* en modo de ejecución (*PlayMode*), con el fin de reproducir y diagnosticar de forma fiable los errores.

Este esfuerzo adicional desplazó ligeramente el calendario de las fases posteriores. Sin embargo, al validar cada incremento por separado, fue posible reorganizar las tareas y recuperar el ritmo sin descartar objetivos. De hecho, la holgura obtenida permitió añadir una funcionalidad de evasión local entre agentes y obstáculos dinámicos que no estaba prevista inicialmente.

La Figura 4.2 recoge el diagrama de Gantt actualizado con la ejecución real del proyecto. Destacan dos diferencias con respecto a la planificación inicial. La primera es la aparición de una fase de robustez geométrica, dedicada a resolver el *clipping*, que desplazó las etapas posteriores. Adicionalmente se incorporó el incremento no planificado de evasión local. En conjunto, estas desviaciones prolongaron el proyecto una semana, hasta las quince, y retrasaron el hito del producto mínimo viable de la semana 10 a la 12.

En esta ocasión los hitos son: **H1** (semana 3), cierre del estudio del estado de la cuestión; **H2** (semana 7), finalización de la estructura de datos volumétrica; **H3** (semana 12), cálculo de rutas robusto una vez resuelto el *clipping*, que consolida el producto mínimo viable; **H4** (semana 14), integración y optimización completadas; y **H5** (semana 15), entrega de la memoria.

Este sobreesfuerzo se traslada también al presupuesto. La dedicación real ascendió a unas **400** horas, frente a las 360 previstas, lo que, manteniendo el resto de partidas, eleva el coste total a 4.892,48 €, tal y como recoge la Tabla 4.2.

Fase	Marzo		Abril				Mayo				Junio				Jul.
Semana	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Estado del arte y estudio previo															
Diseño de la arquitectura															
Estructura de datos volumétrica															
Búsqueda y postprocesado de rutas															
Robustez geométrica (<i>clipping</i>)															
Integración en Unity															
Evaluación y optimización															
Evasión local (no planificada)															
Validación y pruebas (CI)															
Redacción de la memoria															
Hitos															

Figura 4.2: Diagrama de Gantt de la ejecución real del proyecto. Cada columna representa una semana. En gris se resaltan las desviaciones respecto al plan inicial (la fase de robustez geométrica y el incremento no planificado de evasión local); en rosado, las fases que se ajustaron a lo previsto; y en azul, las tareas transversales. Los rombos (◆) señalan los hitos.

Concepto	Coste
Recursos humanos (400 h × 11,43 €/h)	4.572,00 €
Amortización del equipo de desarrollo	87,50 €
Licencias de software	0,00 €
Costes indirectos (5 %)	232,98 €
Total	4.892,48 €

Tabla 4.2: Presupuesto final del proyecto tras el seguimiento.

Capítulo 5

Desarrollo

Este capítulo constituye el núcleo técnico de la memoria. En él se describe, de manera pormenorizada, el proceso de construcción del sistema de navegación volumétrica siguiendo la metodología iterativa e incremental expuesta en la Sección 4.1. Cada una de las secciones que siguen se corresponde con una iteración del desarrollo y se articula en torno a un mismo esquema: el objetivo perseguido, las decisiones de diseño junto con su implementación, las particularidades técnicas más destacables y la validación del incremento resultante.

El recorrido se inicia con una iteración preparatoria que asienta la arquitectura modular y el entorno de trabajo (Iteración 0). Sobre ella se erige un producto mínimo viable, constituido por la representación y la construcción del espacio navegable junto con una primera versión funcional de la búsqueda de rutas (Iteraciones 1 a 3), que se amplía después de forma progresiva hasta alcanzar un producto completo y optimizado. En coherencia con la planificación descrita en la Sección 4.2, cada una de las fases de implementación allí previstas se materializa aquí a través de una o varias iteraciones. Así, una vez alcanzado el núcleo mínimo, las iteraciones restantes abordan sucesivamente el postprocesado de trayectorias, la persistencia en disco, la robustez geométrica, la integración en el motor, la optimización del rendimiento y, finalmente, la evasión local entre agentes. El capítulo concluye con una discusión global que sintetiza la arquitectura resultante y las principales decisiones de diseño.

5.1 Iteración 0: Cimientos del proyecto

La primera iteración no produce funcionalidad de navegación, sino que establece los cimientos sobre los que se sustentará todo el desarrollo posterior. Su objetivo es doble: definir una arquitectura modular que favorezca el desacoplamiento y poner en marcha el flujo de trabajo y las garantías de calidad descritas en el capítulo anterior.

El sistema se organiza en módulos con responsabilidades bien delimitadas, separados físicamente mediante *assemblies* (las unidades de compilación de C#). Esta separación no es

meramente organizativa: al aislar el código de edición y de pruebas en *assemblies* independientes, se garantiza que estos queden excluidos de las compilaciones finales del producto, sin penalizar el tamaño ni el rendimiento del ejecutable distribuido. La estructura resultante se divide en tres grandes bloques, ilustrados en la Figura 5.1:

- **Runtime:** contiene la totalidad de la lógica de navegación (representación del espacio, construcción, búsqueda de rutas y evasión). Es el único módulo que se incluye en la versión final del producto.
- **Editor:** alberga las herramientas de configuración y visualización integradas en el editor de Unity. Depende del módulo *Runtime*, pero nunca a la inversa.
- **Tests:** agrupa las pruebas automatizadas, divididas a su vez en pruebas en modo de edición (*EditMode*) y en modo de ejecución (*PlayMode*).

Internamente, el módulo *Runtime* se subdivide a su vez en espacios de nombres cohesivos: *Core* (las estructuras de datos fundamentales), *Builder* (la construcción del volumen), *Pathfinding* (la búsqueda y el postprocesado), *Avoidance* (la evasión local) y *Unity* (los componentes que exponen el sistema al motor).

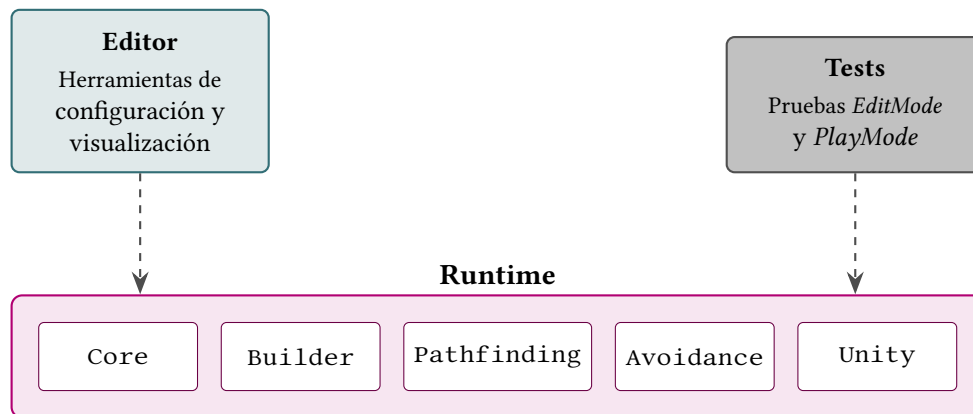


Figura 5.1: Arquitectura modular del sistema y sus dependencias. Los módulos *Editor* y *Tests* dependen del módulo *Runtime* (flechas discontinuas), pero nunca a la inversa; únicamente *Runtime* se incluye en la versión final del producto.

Sobre estos cimientos se configuró el flujo de trabajo basado en *pull requests* y el sistema de CI, descritos en la Sección 4.1. Una de las primeras garantías de calidad que ejecuta este sistema de CI es la verificación automática del estilo: ante cada *pull request*, una comprobación de formato (*format check*) con el formateador CSharpier impide la integración del código si este no se ajusta a la guía de estilo común adoptada. De este modo, la deuda técnica asociada al formato y la consistencia del código se mantuvo controlada, sin intervención manual, a lo largo de todo el proyecto.

5.2 Iteración 1: Representación del espacio navegable

El objetivo de esta iteración es implementar la estructura de datos que representa el espacio navegable de forma compacta. Tal y como se justificó en la Sección 2.3.1, la técnica seleccionada es el **SVO**, una variante del **octree** que solo subdivide las regiones del espacio que contienen geometría.

5.2.1 Organización jerárquica

El **SVO** se modela como un conjunto de capas (*layers*), donde cada capa es un **vector plano** de nodos ordenado por su código de Morton. Las capas superiores representan nodos de gran tamaño (espacio mayoritariamente libre), mientras que las inferiores aumentan progresivamente la resolución. La capa más baja la constituyen las **hojas**, que no almacenan ocho hijos como los nodos internos, sino una rejilla de $4 \times 4 \times 4$ **vóxeles** (véase la Figura 5.2).

Esta decisión de diseño es deliberada y responde a los principios del **DOD** expuestos en la Sección 2.1.4. Almacenar cada hoja como una rejilla densa de 64 **vóxeles**, en lugar de seguir subdividiendo el árbol cuatro niveles más, reduce drásticamente el número de nodos y, sobre todo, mantiene los datos contiguos en memoria. La búsqueda de un nodo dentro de una capa se resuelve mediante una **búsqueda binaria** sobre el vector ordenado, aprovechando que los códigos de Morton preservan la localidad espacial.

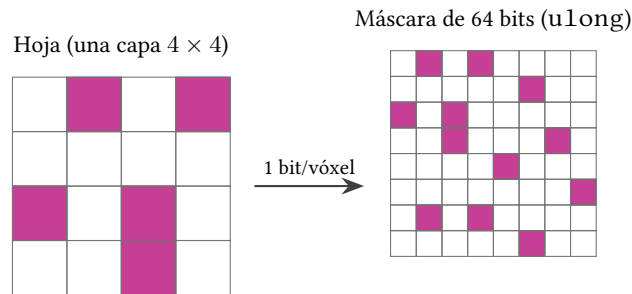


Figura 5.2: Hoja del **SVO** y su máscara de ocupación de 64 bits. Cada hoja apila cuatro capas de 4×4 vóxeles; cada uno de esos $4 \times 4 \times 4 = 64$ vóxeles se corresponde con un bit de un único entero de 64 bits (**ulong**): los ocupados (en rosa) fijan su bit a 1 y los libres a 0.

5.2.2 Codificación de las hojas mediante manipulación de bits

Cada hoja almacena su ocupación en un **único entero de 64 bits (ulong)**, donde cada bit indica si el **vóxel** correspondiente está libre u ocupado. Esta codificación, que materializa la técnica de manipulación de bits introducida en los fundamentos, permite que una hoja completa ocupe tan solo 8 bytes y que operaciones globales como “¿está la hoja completamente

vacía?” o “¿está completamente llena?” se resuelvan con una única comparación entera, sin necesidad de inspeccionar los 64 **vóxels** de forma individual.

La conversión entre las coordenadas tridimensionales de un **vóxel** dentro de la hoja y su índice de bit se realiza también mediante desplazamientos, empaquetando las tres coordenadas (de 2 bits cada una) en los 6 bits del índice.

5.2.3 Códigos de Morton

Para mapear las coordenadas tridimensionales (x, y, z) de un nodo a un índice escalar que preserve la localidad espacial se emplean los códigos de Morton, basados en las curvas de orden Z descritas en la Sección 2.3.1 (véase la Figura 2.3). La codificación se obtiene intercalando los bits de las tres coordenadas mediante una secuencia de operaciones de desplazamiento y máscara, tal y como se muestra en la Figura 5.3.

```

1 // Intercala dos ceros entre cada bit del valor de entrada.
2 static uint Interleave00(uint x)
3 {
4     x = (x ^ (x << 16)) & 0xFF0000FF;
5     x = (x ^ (x << 8)) & 0x0300F00F;
6     x = (x ^ (x << 4)) & 0x030C30C3;
7     x = (x ^ (x << 2)) & 0x09249249;
8     return x;
9 }
10
11 // El código final entrelaza los bits de las tres coordenadas.
12 public MortonCode(int x, int y, int z) =>
13     _code = Interleave00((uint)x)
14         | (Interleave00((uint)y) << 1)
15         | (Interleave00((uint)z) << 2);

```

Figura 5.3: Codificación de un código de Morton mediante intercalado de bits.

Sobre esta representación, operaciones jerárquicas habituales se vuelven triviales: el código del nodo padre se obtiene desplazando cada coordenada un bit a la derecha, y el de un hijo concreto, desplazándola a la izquierda e incorporando el bit correspondiente al octante. Dado que la codificación emplea enteros de 32 bits, la resolución máxima es de 1024 (2^{10}) celdas por eje, lo que acota el número máximo de capas del árbol.

5.2.4 Punteros compactos: el enlace del SVO

Un último elemento fundamental de esta iteración es el mecanismo de referencia entre nodos. En lugar de emplear referencias de C# (punteros de 8 bytes que dispersarían los datos

en la [heap](#)), se diseñó un **puntero compacto** de 32 bits que codifica, en un único entero, toda la información necesaria para localizar cualquier nodo o [vóxel](#) del árbol. El reparto de bits se muestra en la Figura 5.4.

```

1 const uint _IS_NODE_MASK      = 1u << 31;          // bit 31:    tipo
   (1 nodo, 0 voxel)
2 const uint _OFFSET_MASK      = 0x1FFFFFF << 6;    // bits 30..6:
   offset (25 bits)
3 const uint _CONCRETE_DATA_MASK = 0x3F;            // bits 5..0:
   capa o subnodo (6 bits)

```

Figura 5.4: Disposición de los 32 bits del enlace (SVOLink).

El bit más significativo distingue si el enlace apunta a un nodo interno o a un [vóxel](#); los 25 bits centrales almacenan el desplazamiento dentro de la capa o del vector de hojas; y los 6 bits inferiores reutilizan su significado según el tipo: la capa del nodo o el índice del subnodo dentro de la hoja. Esta compacidad no solo ahorra memoria, sino que permite que el enlace se utilice directamente como clave en las colecciones *hash* del buscador de rutas. Para que esto resulte eficiente, el tipo implementa la interfaz `IEnumerable`, evitando así el *boxing* y la comparación por reflexión en la ruta más caliente del algoritmo.

5.3 Iteración 2: Construcción y rasterización del volumen

Definida la estructura de datos, esta iteración aborda su construcción a partir de la geometría de una escena. Este proceso, conocido como rasterización, consiste en consultar qué regiones del espacio están ocupadas por obstáculos para poblar el [SVO](#). Constituye una de las fases más costosas del sistema, por lo que su diseño está dominado por consideraciones de rendimiento.

5.3.1 Muestreo del espacio mediante consultas físicas paralelizadas

Tal y como se anticipó en los fundamentos, el sistema no lee directamente los datos de las mallas, sino que delega la detección de geometría en el motor de físicas a través de consultas `OverlapBox`, que comprueban si una caja determinada interseca con algún *collider*. Dado el ingente número de consultas requeridas, estas se agrupan en lotes (*batches*) mediante la estructura `OverlapBoxCommand` y se ejecutan en paralelo sobre varios hilos secundarios gracias al *Job System* y al compilador *Burst* descritos en la Sección 2.1.4.

La construcción se realiza con una estrategia **de lo grueso a lo fino** (*coarse-to-fine*). En lugar de muestrear de golpe la totalidad del espacio a máxima resolución, el barrido comienza en una capa cercana a la raíz y, en cada nivel, solo subdivide aquellas celdas que contienen

geometría, descartando subárboles enteros vacíos (véase la Figura 5.5). Este descarte temprano es la primera de las dos grandes optimizaciones de la fase de construcción.

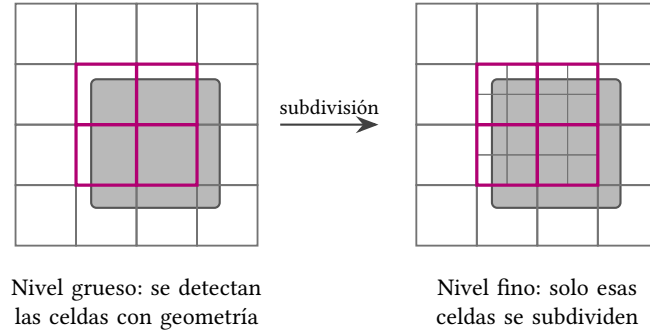


Figura 5.5: Barrido jerárquico de lo grueso a lo fino durante la rasterización. Las celdas vacías se descartan por completo; únicamente las que contienen geometría (resaltadas en rosa) se subdividen en el siguiente nivel de resolución.

5.3.2 Optimización: descarte de hojas vacías

La segunda optimización, y una de las más relevantes del sistema, se aplica durante la rasterización de las hojas. Comprobar la ocupación de una hoja completa exige, en principio, lanzar 64 consultas `OverlapBox`, una por cada **vóxel**. Sin embargo, este coste es casi siempre innecesario.

La razón es estructural: en un entorno de navegación tridimensional típico, el espacio navegable es **mayoritariamente aire libre**. Los obstáculos (paredes, edificios, terreno) ocupan una fracción pequeña del volumen total, por lo que la inmensa mayoría de las hojas no contienen geometría alguna. Esta es, precisamente, la observación que justifica el uso de una representación dispersa.

Para explotarla, la rasterización de hojas se divide en dos pasadas, ilustradas en la Figura 5.7 y esquematizadas en la Figura 5.6:

1. **Pasada gruesa de descarte:** se lanza una **única** consulta `OverlapBox` que cubre la totalidad de la hoja. Esta caja se calcula como la caja envolvente exacta de los 64 vóxeles (expandidos por el radio del agente), de modo que es imposible que una hoja que no intersecte aquí contenga algún vóxel ocupado. Si la consulta no detecta nada, la hoja se deja vacía sin más.
2. **Pasada fina:** únicamente las hojas supervivientes, es decir, aquellas en las que la pasada gruesa sí detectó geometría, pagan el coste de las 64 consultas individuales que determinan qué vóxeles concretos están ocupados.

Dado que las supervivientes son una minoría, esta optimización reduce el número de consultas físicas en un factor cercano a 64 para la inmensa mayoría del volumen, que es vacío. El resultado de la pasada fina se condensa en la máscara de 64 bits de cada hoja mediante un *job* de reducción dedicado.

```

1 // --- Pasada gruesa: una consulta por hoja completa ---
2 lanzar BuildCoarseLeafCommands -> comandos_gruesos (1 por hoja)
3 ejecutar OverlapBox (en lote) -> resultados_gruesos
4 supervivientes = { i : resultados_gruesos[i] golpea algo }
5
6 si no hay supervivientes:
7     todas las hojas quedan vacias // caso muy comun
8
9 // --- Pasada fina: solo sobre las supervivientes ---
10 lanzar BuildLeafCommands(supervivientes) -> comandos (64 por
    superviviente)
11 ejecutar OverlapBox (en lote) -> resultados
12 lanzar ReduceLeafBitmasks -> mascara[superviviente]
13
14 hojas[*] = Vacía
15 para cada j en supervivientes:
16     hojas[j] = DesdeBits(mascara[j])

```

Figura 5.6: Rasterización de hojas en dos pasadas: descarte grueso y muestreo fino.

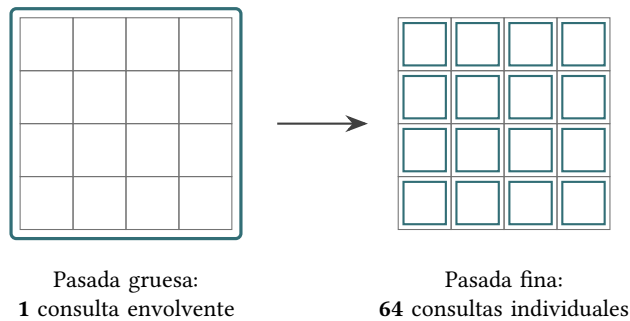


Figura 5.7: Pasada gruesa de descarte frente a pasada fina de muestreo. La pasada gruesa resuelve la hoja entera con una única consulta envolvente; solo si esta detecta geometría se ejecuta la pasada fina, con una consulta por vóxel (64 en una hoja de $4 \times 4 \times 4$).

5.3.3 Enlazado de la vecindad

Una vez rasterizadas las hojas y construidas las capas superiores, resta una última etapa: enlazar cada nodo con sus seis vecinos (en las direcciones $\pm X$, $\pm Y$ y $\pm Z$), información imprescindible para que el buscador de rutas pueda navegar el grafo.

El enlazado se resuelve recorriendo las capas de la raíz hacia las hojas y aplicando una regla con **herencia del padre**. Para cada nodo y dirección, se busca primero si existe un vecino del mismo tamaño (mismo nivel) mediante el código de Morton correspondiente. Si no existe, en lugar de buscar exhaustivamente vecinos de distinto tamaño, el nodo simplemente **hereda el enlace del vecino de su padre** (un nodo de mayor tamaño). Esta solución, ilustrada en la Figura 5.8, resuelve de forma elegante y eficiente la transición entre regiones de distinta resolución, que es uno de los aspectos más delicados de la navegación sobre estructuras jerárquicas.

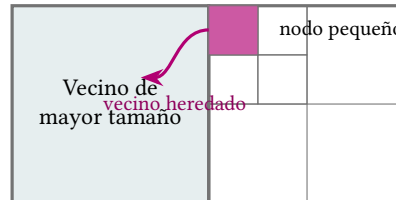


Figura 5.8: Enlazado de la vecindad mediante herencia del nodo padre. Cuando un nodo pequeño carece de un vecino de su mismo tamaño en una dirección, hereda como vecino el nodo de mayor tamaño adyacente a su padre, evitando una búsqueda exhaustiva entre celdas de distinto tamaño.

Al término de esta iteración el sistema es ya capaz de transformar una escena tridimensional en un **SVO** completamente enlazado y navegable.

5.4 Iteración 3: Búsqueda de rutas tridimensional

Esta iteración implementa el corazón del sistema y completa el producto mínimo viable: la búsqueda de una trayectoria libre de colisiones entre dos puntos. Conforme a lo expuesto en los fundamentos, se adapta el algoritmo A^* [9] para operar sobre el **SVO**.

5.4.1 Búsqueda sobre un grafo de resolución heterogénea

La particularidad esencial de este buscador, frente a una implementación clásica de A^* sobre una rejilla uniforme, es que el grafo subyacente posee una **resolución heterogénea**: un mismo salto puede atravesar un nodo enorme de espacio abierto o un diminuto **vóxel** junto a un obstáculo. El algoritmo gestiona esta heterogeneidad combinando tres comportamientos durante la expansión de vecinos, ilustrados en la Figura 5.9:

- Si un vecino es un nodo grande de **espacio libre**, se trata como un único salto, lo que permite cruzar grandes extensiones vacías con muy pocas expansiones.
- Si un vecino es un nodo que **contiene geometría**, no puede tratarse como un salto barato y seguro: se expande en sus ocho hijos para examinarlos a mayor resolución.
- Si un vecino es una **hoja parcialmente ocupada**, se expande directamente en sus **vóxeles** libres, descartando los ocupados.

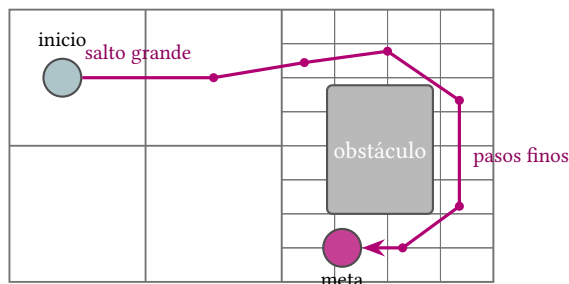


Figura 5.9: Expansión de A* sobre un grafo de resolución heterogénea. En el espacio abierto la búsqueda avanza con saltos grandes entre nodos de gran tamaño; al aproximarse al obstáculo, se ramifica en nodos y vóxeles cada vez más pequeños para rodearlo con precisión.

```

1 empujar(inicio, f = h(inicio))
2 mientras lista_abierta no vacia:
3     n = extraer_minimo()
4     si cerrado[n]: continuar
5     marcar cerrado[n]
6     si n == meta: devolver reconstruir(n)
7
8     para cada m en vecinos(n):
9         si m es nodo grande con geometria: expandir sus hijos;
            continuar
10        si m es hoja parcialmente ocupada: expandir sus voxeles
            libres; continuar
11        si bloqueado(m): continuar
12
13        g_nuevo = g[n] + coste(n, m)
14        si g_nuevo < g[m]:
15            g[m] = g_nuevo; procedencia[m] = n
16            empujar(m, g_nuevo + h(m))

```

Figura 5.10: Esquema del bucle principal de la búsqueda A* adaptada.

5.4.2 Función de coste y heurística

El sistema ofrece dos modos de medir el coste de la trayectoria. El modo basado en **distancia euclídea** computa el coste real entre los centros de los nodos, produciendo rutas geométricamente más cortas. El modo basado en **conteo de nodos**, en cambio, asigna un coste unitario a cada salto con independencia del tamaño del nodo; al penalizar por igual cualquier salto, este modo sesga la búsqueda hacia los nodos grandes y, por tanto, hacia el espacio abierto, alejando al agente de los obstáculos.

La heurística empleada es la distancia en línea recta hasta la meta, ponderada por un peso configurable W (heurística voraz). Para el modo de conteo de nodos, dado que el coste g no se expresa en unidades de distancia, la heurística se reescala dividiéndola por el tamaño del nodo más grande, garantizando así que ambos términos de la función $f(n) = g(n) + W \cdot h(n)$ sean comparables.

5.4.3 Reconstrucción y robustez

Cuando la búsqueda alcanza la meta, la trayectoria se reconstruye recorriendo hacia atrás los enlaces de procedencia almacenados para cada nodo. El buscador contempla además los casos límite, como volúmenes completamente vacíos (en los que la ruta es trivial), extremos no válidos o situados sobre geometría, y un presupuesto máximo de nodos que acota el coste de las búsquedas sin solución.

Al completar esta iteración se dispone del producto mínimo viable comprometido en la planificación: un sistema capaz de construir el volumen y calcular una ruta sobre él. No obstante, tal y como se documentó en el seguimiento de la planificación (Sección 4.3), garantizar la robustez geométrica de estas rutas resultaría más costoso de lo previsto, cuestión que se aborda en la Iteración 6.

5.5 Iteración 4: Postprocesado de trayectorias

Las rutas producidas por la búsqueda sobre el volumen discretizado resultan artificiales, al estar compuestas por multitud de segmentos alineados con la rejilla. Esta iteración implementa el postprocesado en dos fases descrito en los fundamentos, con el fin de dotar al agente de un movimiento orgánico.

5.5.1 Atajo voraz mediante línea de visión

La primera fase aplica un algoritmo de *string pulling* en su variante voraz, que elimina los nodos intermedios redundantes. Partiendo del primer punto, el algoritmo intenta conectarlo

con el punto más lejano posible siempre que exista **línea de visión directa** entre ambos, descartando los nodos intermedios.

La comprobación de la línea de visión es un punto crítico de precisión y rendimiento. Se resuelve mediante el algoritmo DDA [37], que recorre de forma incremental los **vóxeles** atravesados por el segmento, comprobando si alguno está ocupado. El recorrido avanza, en cada paso, por el eje cuyo siguiente plano de **vóxel** se encuentra más próximo, evitando así tanto omitir **vóxeles** como evaluarlos por duplicado. La Figura 5.11 resume el procedimiento.

```

1 celda  = floor(inicio)           // voxel inicial
2 paso   = signo(direccion)       // +1 o -1 por eje
3 tDelta = 1 / |direccion|        // avance parametrico por eje
4 tCruce = distancia al primer plano de voxel por eje
5
6 mientras dentro de la rejilla:
7     si ocupado(celda): devolver false // obstaculo: sin linea
   de vision
8
9     eje = aquel con menor tCruce      // siguiente plano mas
   cercano
10    si tCruce[eje] >= longitud_segmento: terminar
11    celda[eje] += paso[eje]
12    tCruce[eje] += tDelta[eje]
13
14 devolver true

```

Figura 5.11: Recorrido DDA para la comprobación de línea de visión.

5.5.2 Suavizado mediante *splines* de Catmull-Rom

La trayectoria depurada se suaviza interpolando sus puntos con un *spline* de Catmull-Rom, evaluado a una resolución fija entre cada par de puntos de control. Como se argumentó en los fundamentos (véase la Figura 2.4), la elección de esta curva se debe a que **pasa de forma exacta por todos los puntos de control**. Esta propiedad es esencial para la seguridad de la trayectoria: dado que la fase anterior garantiza que dichos puntos se hallan en espacio libre, obligar a la curva a cruzar exactamente por ellos evita que el suavizado introduzca colisiones accidentales con la geometría. La evaluación de la curva se muestra en la Figura 5.12.

```

1 static Vector3 EvaluateCatmullRom(Vector3 p0, Vector3 p1, Vector3
  p2, Vector3 p3, float t) =>
2     0.5f * (
3         2f * p1
4         + (-p0 + p2) * t
5         + (2f * p0 - 5f * p1 + 4f * p2 - p3) * (t * t)
6         + (-p0 + 3f * p1 - 3f * p2 + p3) * (t * t * t)
7     );

```

Figura 5.12: Evaluación de un segmento del *spline* de Catmull-Rom.

5.6 Iteración 5: Persistencia e integridad de la escena

Reconstruir el *SVO* cada vez que se carga la escena sería prohibitivo en tiempo de ejecución. Esta iteración implementa el proceso de *bake* descrito en los fundamentos: la construcción anticipada del volumen desde el editor y su almacenamiento en disco para una carga posterior inmediata.

5.6.1 Serialización en un único bloque binario

El *SVO* se serializa en un **único bloque binario** (*blob*) que se almacena dentro de un *asset* del motor. Cada hoja se vuelca como su entero de 64 bits, y cada nodo como nueve enteros de 32 bits (su código de Morton, los enlaces a su primer hijo y a su padre, y sus seis vecinos). Esta representación plana y sin referencias es a la vez compacta y extremadamente rápida de cargar, ya que la reconstrucción se reduce a una lectura secuencial del flujo de bytes, sin necesidad de recalcular la vecindad ni la jerarquía.

5.6.2 Integridad de la escena: reinterpretación de los bits de un *float*

Un riesgo evidente del *bake* es que la escena se modifique tras él, dejando los datos almacenados obsoletos. Para detectarlo, el sistema calcula un *hash* de la geometría relevante en el momento del *bake* y lo compara con el de la escena actual al cargar. Conforme se justificó en los fundamentos, se emplea el algoritmo *FNV-1a* [42] en lugar del `GetHashCode` por defecto, ya que este último no es determinista entre ejecuciones [41].

El algoritmo *FNV-1a* opera sobre los **bytes** de los datos, lo que plantea un reto al alimentarlo con valores en coma flotante (las posiciones, rotaciones y escalas de los obstáculos): es necesario obtener el patrón de bits **exacto** de cada `float`, y no una conversión numérica que truncaría o redondearía su valor. Sin embargo, C# no permite reinterpretar directamente los bits de un tipo como los de otro.

La solución implementada recurre a una **unión** al estilo del lenguaje C, ilustrada en la Figura 5.13. Mediante el atributo `[StructLayout(LayoutKind.Explicit)]` se define una estructura con dos campos, un `float` y un `int`, forzados a compartir la misma posición de memoria con `[FieldOffset(0)]`. Al escribir el valor en el campo `float` y leer acto seguido el campo `int`, se obtiene el entero cuya representación binaria coincide bit a bit con la del `float` original. Esta técnica es eficiente (no realiza reservas de memoria ni saltos condicionales) y garantiza el determinismo necesario para que el *hash* sea fiable entre distintas sesiones.

```

1 [StructLayout(LayoutKind.Explicit)]
2 struct FloatIntUnion
3 {
4     [FieldOffset(0)] public float Float;
5     [FieldOffset(0)] public int   Int;
6 }
7
8 // Devuelve el int cuya representacion de bits coincide con la del
9   float.
10 public static int FloatBitsAsInt(float value) =>
11     new FloatIntUnion { Float = value }.Int;

```

Figura 5.13: Reinterpretación de los bits de un `float` como un `int` mediante una unión.

Para que el *hash* no dependa del orden arbitrario en que el motor devuelve los *colliders*, estos se ordenan previamente por una huella estable (posición, rotación y escala) antes de alimentar el algoritmo.

5.7 Iteración 6: Robustez geométrica y radio del agente

Esta iteración aborda directamente la desviación más significativa de la planificación, anticipada en la Sección 4.3: la consolidación de la robustez geométrica de las rutas. Las primeras versiones del buscador producían, en ocasiones, trayectorias que rozaban o atravesaban la geometría del entorno (un fenómeno conocido como *clipping*), especialmente en las inmediaciones de las aristas y esquinas de los obstáculos.

5.7.1 Consideración del radio del agente

La causa principal residía en que el sistema trataba al agente como un punto adimensional. La solución consiste en **inflar** los obstáculos por el radio del agente durante la rasterización: al expandir cada caja de consulta `OverlapBox` por dicho radio, todo **vóxel** situado a una

distancia de la geometría inferior al tamaño del agente se marca como ocupado. De este modo, cualquier ruta que el buscador considere libre garantiza, por construcción, el espacio de holgura necesario para que el agente no colisione. La Figura 5.14 ilustra el problema y su corrección.

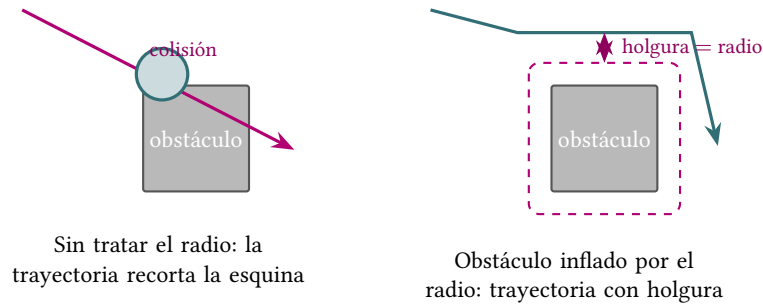


Figura 5.14: Corrección del *clipping* mediante el inflado de obstáculos por el radio del agente. Si no se considera el tamaño del agente, una trayectoria que recorta una esquina provoca colisión (izquierda). Al inflar el obstáculo por el radio del agente (margen discontinuo), la trayectoria conserva siempre la holgura necesaria (derecha).

5.7.2 Validación intensiva mediante pruebas

Diagnosticar de forma fiable este tipo de errores geométricos, intermitentes y dependientes de la configuración concreta de la escena, exigió el desarrollo de una batería de pruebas considerablemente más extensa de la prevista. A las pruebas en modo de edición se sumaron pruebas en modo de ejecución (*PlayMode*), capaces de instanciar escenas completas, construir el volumen y validar la navegabilidad de las trayectorias resultantes contra el motor de físicas real. Tanto estas pruebas en modo de ejecución como las pruebas en modo de edición existentes se ejecutan de forma automática en el sistema de CI ante cada *pull request*, de modo que cualquier regresión queda detectada antes de integrar los cambios en la rama principal. Este esfuerzo de validación, aunque desplazó ligeramente el calendario, resultó determinante para consolidar un núcleo fiable sobre el que sustentar el resto de funcionalidades.

5.8 Iteración 7: Integración en Unity y telemetría

Con un núcleo robusto, esta iteración se centra en la usabilidad, materializando el sistema en herramientas accesibles tanto para perfiles técnicos como para diseñadores y artistas.

5.8.1 Componentes y modos de construcción

El sistema se expone principalmente a través del componente `NavVolumeSpace`, que encapsula un volumen de navegación y ofrece tres modos de construcción para adaptarse a

distintos flujos de trabajo: **precalculado** (*Baked*), que carga los datos serializados; **construido al inicio** (*BuildOnAwake*); y **manual**. A este se añaden los componentes que representan a las entidades de la escena, como los agentes y los obstáculos dinámicos.

5.8.2 Visualización mediante Gizmos

Para facilitar la comprensión y la depuración del sistema, se desarrolló una visualización del **SVO** en la vista de escena mediante Gizmos. Esta representa la jerarquía del árbol con un código de colores que distingue los nodos vacíos, los que contienen geometría, las hojas parciales y los **vóxels** ocupados (véase la Figura 5.15). Dado que un volumen puede contener cientos de miles de nodos, el dibujado incorpora dos salvaguardas de rendimiento: un **presupuesto máximo** de primitivas y un **descarte por distancia** a la cámara, que evita representar el detalle fino de regiones lejanas.

[Figura pendiente de incluir]

Captura de la vista de escena de Unity mostrando la visualización del **SVO** mediante Gizmos sobre un escenario con obstáculos: nodos de distintos tamaños coloreados según su estado (vacíos, con geometría, hojas parciales) y los vóxeles ocupados resaltados.

Figura 5.15: Visualización del **SVO** en la vista de escena mediante Gizmos.

5.8.3 Telemetría: el cálculo de la memoria estimada

El inspector incorpora un panel de estadísticas que ayuda al usuario a dimensionar y ajustar el volumen (Figura 5.17). Entre las métricas más relevantes figuran el ahorro de **vóxels** frente a una representación densa y una **estimación de la memoria** ocupada por la estructura.

El cálculo de esta memoria estimada se fundamenta en los principios de disposición de objetos en la memoria administrada de C#. En una plataforma de 64 bits, todo objeto reservado en la **heap** acarrea una sobrecarga: una cabecera de objeto (que incluye el puntero a su tabla de métodos y el bloque de sincronización) y, en el caso de los vectores, espacio adicional para almacenar su longitud. A partir de estas constantes aproximadas, la estimación recorre la estructura sumando, para cada vector de nodos y de hojas, su cabecera más el tamaño real de cada elemento (obtenido con `Unsafe.SizeOf`) multiplicado por su número de elementos, tal y como se muestra en la Figura 5.16.

```

1 const int _OBJECT_HEADER_BYTES = 24; // cabecera de objeto
2 const int _ARRAY_HEADER_BYTES = 24; // cabecera de objeto +
   longitud
3 const int _REFERENCE_BYTES      = 8;  // tamaño de una referencia
4
5 int total = _OBJECT_HEADER_BYTES;
6
7 // Vector de hojas: cada hoja es un ulong (8 bytes).
8 total += _ARRAY_HEADER_BYTES + Unsafe.SizeOf<SVOLeaf>() *
   leaves.Length;
9
10 // Vector externo de capas (un vector dentado de referencias).
11 total += _ARRAY_HEADER_BYTES + _REFERENCE_BYTES * layers.Length;
12
13 // Cada capa: su cabecera mas el tamaño real de cada nodo.
14 foreach (var layer in layers)
15     total += _ARRAY_HEADER_BYTES + Unsafe.SizeOf<SVONode>() *
   layer.Length;

```

Figura 5.16: Estimación de la memoria ocupada por el SVO.

Conviene subrayar que se trata de una **estimación**, no de una medición exacta: su propósito es ofrecer al usuario una magnitud orientativa y, sobre todo, comparable entre configuraciones. Por su parte, el ahorro de espacio se cuantifica comparando el número de **vóxeles** efectivamente reservados frente al de un árbol completamente denso de la misma profundidad, que crece como $8^{(L-1)}$ hojas para L capas. Esta cifra evidencia de forma tangible el beneficio de la representación dispersa, que constituye el fundamento de todo el sistema.

[Figura pendiente de incluir]

Captura del panel de estadísticas del inspector: tamaño de vóxel, vóxeles reservados frente al total teórico con su porcentaje de ahorro, memoria estimada, y los gráficos de nodos por capa y de ahorro disperso por capa (mapa de calor).

Figura 5.17: Panel de estadísticas del volumen en el inspector del editor.

5.9 Iteración 8: API pública, optimización del rendimiento

Esta iteración consolida la interfaz pública del sistema y acomete la fase de evaluación y optimización del rendimiento prevista en la planificación.

5.9.1 API de navegación y consultas espaciales

Sobre el componente NavVolumeSpace se expone una [API](#) que cubre los casos de uso habituales de la navegación. Más allá del cálculo de rutas, se ofrecen consultas espaciales como la comprobación de si un punto es navegable, el ajuste de un punto al [vóxel](#) navegable más cercano (*snapping*) o la obtención de puntos navegables aleatorios, de utilidad para poblar escenas o dirigir el deambular de los agentes.

El cálculo de rutas se ofrece en dos variantes. La variante **síncrona** reutiliza una única instancia del buscador, mientras que la variante **asíncrona** ejecuta la búsqueda en un hilo secundario, evitando bloquear el hilo principal en escenarios con muchas peticiones. Esta última emplea una reserva (*pool*) de buscadores para no incurrir en reservas de memoria repetidas, y admite la cancelación de la petición mediante un *token*.

5.9.2 Optimizaciones de rendimiento

La fase de optimización, guiada por el perfilado del sistema, introdujo numerosas mejoras de bajo nivel coherentes con los fundamentos del [DOD](#). Entre las más significativas cabe destacar:

- **Reutilización de *buffers*:** el buscador reutiliza las colecciones de su estado interno entre peticiones, en lugar de reservarlas de nuevo, reduciendo la presión sobre el recolector de basura.
- **Comprobación de cancelación amortizada:** en lugar de consultar el *token* de cancelación en cada expansión (una operación relativamente costosa), solo se comprueba periódicamente mediante una máscara de bits.
- **Acceso por referencia de solo lectura:** la consulta de nodos devuelve una referencia (`ref readonly`) en lugar de una copia de la estructura, evitando duplicar datos en las rutas más calientes.
- **Comparación sin *boxing*:** como se adelantó en la Sección [5.2.4](#), los enlaces implementan `IEquatable` para su uso eficiente como clave en las colecciones *hash* del buscador.

Asimismo, se optimizó la propia construcción del volumen, evitando reordenaciones innecesarias de las capas cuando no faltan nodos hermanos, lo que aceleró notablemente el [bake](#) en escenas densas.

5.10 Iteración 9: Evasión local entre agentes

La última iteración incorpora un incremento que, tal y como se documentó en el seguimiento de la planificación (Sección 4.3), no figuraba como objetivo prioritario inicial, pero que la holgura recuperada permitió abordar: la evasión local entre agentes. La búsqueda de rutas descrita hasta ahora calcula trayectorias sobre la geometría estática, pero no evita que varios agentes en movimiento colisionen entre sí.

5.10.1 El algoritmo ORCA

Para resolverlo se implementó el algoritmo **ORCA** [40], introducido en los fundamentos, que razona sobre el espacio de velocidades para calcular, en cada instante, una velocidad segura y próxima a la deseada. La implementación es un port de las rutinas de programación lineal en tres dimensiones de la biblioteca de referencia RVO2-3D [57]. El solucionador busca la velocidad óptima que satisface todas las restricciones (semitplanos de velocidad) y, cuando el problema resulta infactible (demasiadas restricciones simultáneas), recurre a una relajación que minimiza la penetración en las restricciones (véase la Figura 5.18). Dado que este solucionador se ejecuta para cada agente y en cada fotograma, sus estructuras temporales se reservan en el **stack** mediante `stackalloc` y se manipulan a través de `Span<T>`, lo que evita reservas en el **heap** y la consiguiente presión sobre el recolector de basura en una de las rutas más críticas del sistema.

Una particularidad de la integración es que el sistema no solo considera la evasión recíproca entre agentes, sino que también deriva restricciones a partir de los **vóxeles** ocupados del **SVO** cercano, de modo que la evasión local respeta igualmente la geometría estática del entorno.

5.10.2 Integración con el sistema de jobs

Por su naturaleza masivamente paralela (cada agente resuelve su propio problema), la simulación de evasión se integró sobre el *Job System* y Burst. Los agentes vecinos se localizan eficientemente mediante un **hash espacial**, que evita comparar cada agente con todos los demás. Además, los *jobs* se planifican al final del fotograma y se completan al inicio del siguiente, de modo que su cómputo se solapa con el renderizado, minimizando su impacto sobre la tasa de fotogramas.

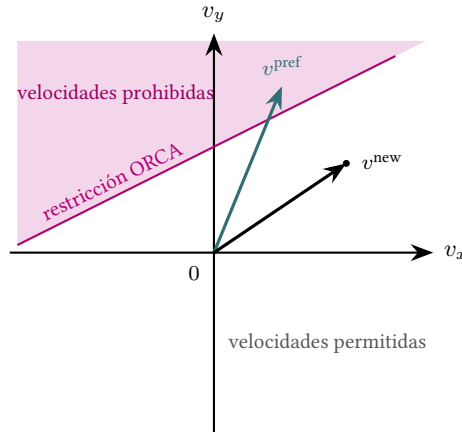


Figura 5.18: Resolución de la velocidad segura en el espacio de velocidades de ORCA. Cada agente vecino y la geometría inducen un semiplano de velocidades prohibidas (zona sombreada). Como la velocidad preferida v^{pref} cae en una región prohibida, el algoritmo escoge la velocidad factible v^{new} más próxima, sobre la frontera de la región segura.

5.11 Discusión global del sistema

Tras recorrer las distintas iteraciones, conviene contemplar el sistema en su conjunto. El resultado es una canalización (*pipeline*) claramente diferenciada en dos grandes momentos, ilustrada en la Figura 5.19.

Por un lado, una fase de **preprocesado en el editor** (el *bake*), que transforma la geometría de la escena en un *SVO* y lo serializa en disco. Esta fase concentra deliberadamente el grueso del coste computacional (las consultas físicas, la construcción de la jerarquía y el enlazado de la vecindad), aprovechando que se ejecuta una sola vez y fuera del tiempo real.

Por otro lado, una fase de **ejecución en tiempo real**, que carga el volumen precalculado de forma inmediata y, sobre él, resuelve peticiones de ruta (búsqueda, atajo y suavizado) y simula la evasión local entre agentes. El diseño de esta fase está dominado por la preocupación por el rendimiento: estructuras de datos compactas y contiguas, paralelización mediante el *Job System* y Burst, y un uso cuidadoso de la memoria que minimiza la intervención del recolector de basura.

A lo largo del desarrollo, varias decisiones de diseño transversales han demostrado su valor. La representación dispersa y la codificación compacta de la información (las máscaras de bits de las hojas y los enlaces de 32 bits) reducen tanto el consumo de memoria como los fallos de caché. La estrategia de lo grueso a lo fino, presente tanto en la rasterización como en la propia búsqueda de rutas, evita sistemáticamente trabajar a máxima resolución donde no es necesario. Y la separación modular del código, establecida desde la primera iteración, ha permitido validar cada incremento de forma aislada, sosteniendo la fiabilidad del conjunto.

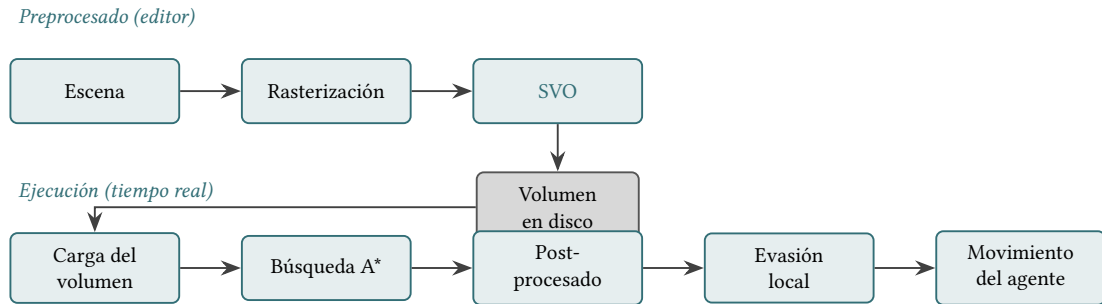


Figura 5.19: Flujo de datos global del sistema, del preprocesado a la ejecución. En el editor, la escena se rasteriza para construir el SVO, que se serializa en disco. En tiempo de ejecución, el volumen se carga y, sobre él, se resuelven las consultas de ruta (búsqueda, postprocesado y evasión local) que culminan en el movimiento del agente.

En definitiva, el sistema resultante satisface los objetivos planteados: representa de forma eficiente el espacio navegable tridimensional, calcula sobre él trayectorias fluidas y libres de colisiones, y lo hace a través de una herramienta integrada en el motor y accesible para el usuario final. El análisis del rendimiento del sistema y las conclusiones derivadas se presentan en los capítulos siguientes.

Lista de acrónimos

- API** Application Programming Interface. 7, 8, 44
- CI** Continuous Integration. 11, 29, 41
- CLR** Common Language Runtime. 9
- CPU** Central Processing Unit. 8
- DAFO** Debilidades, Amenazas, Fortalezas y Oportunidades. i, 5, 21, 22
- DDA** Digital Differential Analyzer. iv, 13, 38
- DOD** Diseño Orientado a Datos. iv, 8, 9, 12, 30, 44
- DOTS** Data Oriented Technology Stack. 8, 9
- ECTS** European Credit Transfer and Accumulation System. 24
- FPS** Fotogramas Por Segundo. 3
- NPC** Non-Playable Character. 1, 19
- ORCA** Optimal Reciprocal Collision Avoidance. 14, 15, 45
- POO** Programación Orientada a Objetos. iv, 6, 8, 9
- SIMD** Single Instruction, Multiple Data. 9
- SVO** Sparse Voxel Octree. iv, v, 12, 17, 24, 30, 32, 35, 39, 42, 43, 45–47
- UI** User Interface. 6

Glosario

A* Algoritmo de búsqueda de caminos en grafo que hace uso de heurísticas. [iv](#), [2](#), [13](#), [17](#), [35](#), [36](#)

AAA Clasificación para videojuegos desarrollados con un alto presupuesto. [20](#)

ad hoc Expresión para referirse a que se hace con un fin determinado. [3](#), [12](#)

bake Proceso de precomputación realizado durante la fase de desarrollo (en el editor). Consiste en calcular de forma anticipada información costosa —como la generación de mallas de navegación, estructuras espaciales o iluminación estática— y almacenarla en disco, evitando así realizar dichos cálculos durante el tiempo de ejecución del sistema. [16](#), [39](#), [44](#), [46](#)

deadlock Situación de interbloqueo temporal o permanente. En el contexto de la navegación de agentes, ocurre cuando dos o más entidades bloquean mutuamente sus respectivas trayectorias, impidiendo que cualquiera de ellas pueda avanzar hacia su destino al verse incapaces de resolver la evasión geométrica de forma local. [15](#)

FNV-1a Función hash no criptográfica diseñada por Fowler, Noll y Vo, conocida por su rapidez y alta tasa de dispersión. Se utiliza frecuentemente en tablas hash y aplicaciones donde el rendimiento es crítico, ya que procesa los datos de forma eficiente mediante operaciones simples a nivel de bits (XOR y multiplicación). [16](#), [39](#)

heap Región de memoria utilizada para la asignación dinámica. Los bloques de memoria se reservan y liberan en tiempo de ejecución, ya sea manualmente o mediante un recolector de basura, sin un orden establecido. [8](#), [10](#), [32](#), [42](#), [45](#)

LLVM Infraestructura de compilación modular diseñada para optimizar el tiempo de compilación, enlace y ejecución de programas. Su nombre no es un acrónimo. [9](#), [21](#)

- no holonómico** Entidad cuyo movimiento está sujeto a restricciones cinemáticas y físicas, lo que le impide cambiar de dirección o velocidad de manera instantánea (por ejemplo, un vehículo con un radio de giro limitado). 18
- octree** Estructura de datos en árbol donde cada nodo interno tiene exactamente ocho hijos. Se utiliza principalmente para la partición jerárquica y eficiente del espacio tridimensional. 3, 12, 30
- race condition** Anomalía que ocurre en la programación concurrente o multihilo cuando dos o más subprocesos intentan acceder y modificar un recurso compartido de forma simultánea. El estado final del sistema se vuelve impredecible, ya que depende del orden exacto e incontrolable en el que el procesador planifique la ejecución de las instrucciones. 8
- stack** Región de memoria gestionada de forma automática y secuencial (último en entrar, primero en salir). Se utiliza para almacenar variables locales y el contexto de las llamadas a funciones durante la ejecución de un programa. 10, 45
- vóxel** Acrónimo de *volumetric pixel* (píxel volumétrico). Constituye la unidad cúbica mínima procesable dentro de una matriz o cuadrícula tridimensional. Es el equivalente tridimensional del píxel y se utiliza ampliamente para discretizar y representar la ocupación de espacios volumétricos. 12, 30–33, 35, 36, 38, 40, 42–45

Bibliografía

- [1] S. Russell, P. Norvig, and A. Intelligence, “A modern approach,” *Artificial Intelligence. Prentice-Hall, Egnlewood Cliffs*, vol. 25, no. 27, pp. 79–80, 1995.
- [2] C. W. Reynolds *et al.*, “Steering behaviors for autonomous characters,” in *Game developers conference*, vol. 1999. San Francisco, CA, 1999, pp. 763–782.
- [3] I. Millington and J. D. Funge, *Artificial intelligence for games*, 2nd ed. Burlington (Massachusetts): Morgan Kaufmann/Elsevier, 2009.
- [4] Z. Abd Algfoor, M. S. Sunar, and H. Kolivand, “A comprehensive study on pathfinding techniques for robotics and video games,” *International Journal of Computer Games Technology*, vol. 2015, no. 1, p. 736138, 2015.
- [5] Epic Games, “Navigation system in unreal engine,” <https://docs.unrealengine.com/5.3/en-US/navigation-system-in-unreal-engine/>, consultado el 17 de junio de 2026.
- [6] Unity Technologies, “AI navigation package documentation,” <https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/index.html>, consultado el 17 de junio de 2026.
- [7] Godot Engine, “Navigationserver3d — godot engine documentation,” https://docs.godotengine.org/en/stable/classes/class_navigationserver3d.html, consultado el 17 de junio de 2026.
- [8] M. Mononen, “Recast navigation,” <https://recastnav.com/>, 2009, consultado el 17 de junio de 2026.
- [9] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

- [10] Unity Technologies, “Creación de un agente de NavMesh — manual de unity,” <https://docs.unity3d.com/es/530/Manual/nav-CreateNavMeshAgent.html>, consultado el 17 de junio de 2026.
- [11] Industrial Inspection & Analysis, “What are the 6 degrees of freedom? (6DoF explained),” <https://industrial-ia.com/what-are-the-6-degrees-of-freedom-6dof-explained/>, consultado el 17 de junio de 2026.
- [12] S. M. LaValle, *Planning algorithms*. Cambridge university press, 2006.
- [13] D. Meagher, “Geometric modeling using octree encoding,” *Computer graphics and image processing*, vol. 19, no. 2, pp. 129–147, 1982.
- [14] D. Brewer, “Getting off the NavMesh: Navigating in fully 3D environments,” <https://www.gdcvault.com/play/1022016/Getting-off-the-NavMesh-Navigating>, 2015, game Developers Conference (GDC). consultado el 17 de junio de 2026.
- [15] Unity Technologies, “Unity,” <https://unity.com/>, 2026, consultado el 17 de junio de 2026.
- [16] —, “Editor tools - unity 2019.3,” 2020, accedido: 15 de junio de 2026. [Online]. Available: <https://unity.com/es/releases/2019-3/editor-tools>
- [17] NVIDIA Corporation, “Nvidia physx system software,” 2019, accedido: 15 de junio de 2026. [Online]. Available: <https://www.nvidia.com/es-es/drivers/physx/physx-9-19-0218-driver/>
- [18] Unity Technologies, “Unity physics api documentation,” 2022, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.physics@6.5/api/index.html>
- [19] —, “Data-oriented technology stack (dots),” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://unity.com/dots>
- [20] D. Albano, “Data processing in c#: Performance of oop vs dod,” 2019, accedido: 15 de junio de 2026. [Online]. Available: <https://medium.com/@d.albano/data-processing-in-c-performance-of-oop-vs-dod-a23e94babf5c>
- [21] Unity Technologies, “C# job system,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Manual/JobSystem.html>
- [22] —, “Burst user guide,” <https://docs.unity3d.com/Packages/com.unity.burst@1.8/manual/index.html>, 2026, consultado el 17 de junio de 2026.

- [23] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2004, pp. 75–86.
- [24] Unity Technologies, “Unity.mathematics package,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.mathematics@1.2/manual/index.html>
- [25] —, “Native collections in unity,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Manual/JobSystemNativeContainer.html>
- [26] Microsoft Corporation, “Visual studio,” <https://visualstudio.microsoft.com/>, 2026, consultado el 17 de junio de 2026.
- [27] Software Freedom Conservancy, “Git,” <https://git-scm.com/>, 2026, consultado el 17 de junio de 2026.
- [28] GitHub, Inc., “Github,” <https://github.com/>, 2026, consultado el 17 de junio de 2026.
- [29] —, “Github actions,” <https://github.com/features/actions>, 2026, consultado el 17 de junio de 2026.
- [30] CSharpier Contributors, “Csharpier — an opinionated code formatter for C#,” <https://csharpier.com/>, 2026, consultado el 17 de junio de 2026.
- [31] Unity Technologies, “Unity test framework,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://docs.unity3d.com/Packages/com.unity.test-framework@1.4/manual/index.html>
- [32] E. Emerson, “Crowd pathfinding and steering using flow field tiles,” in *Game AI Pro 360: Guide to Movement and Pathfinding*. CRC Press, 2019, pp. 67–76.
- [33] G. M. Morton, *A computer oriented geodetic data base and a new technique in file sequencing*. International Business Machines Company, 1966.
- [34] M. Vinkler, J. Bittner, and V. Havran, “Extended morton codes for high performance bounding volume hierarchy construction,” in *Proceedings of High Performance Graphics*, ser. HPG ’17. Association for Computing Machinery, 2017. [Online]. Available: https://www.highperformancegraphics.org/wp-content/uploads/2017/Papers-Session3/HPG207_ExtendedMortonCodes.pdf
- [35] E. W. Dijkstra, “A note on two problems in connexion with graphs,” in *Edsger Wybe Dijkstra: his life, work, and legacy*, 2022, pp. 287–290.

- [36] D. Harabor and A. Grastien, “An optimal any-angle pathfinding algorithm,” in *Proceedings of the International Conference on Automated Planning and Scheduling*, vol. 23, 2013, pp. 308–311.
- [37] J. Amanatides, A. Woo *et al.*, “A fast voxel traversal algorithm for ray tracing,” in *Eurographics*, vol. 87, no. 3, 1987, pp. 3–10.
- [38] E. Catmull and R. Rom, “A class of local interpolating splines,” in *Computer aided geometric design*. Elsevier, 1974, pp. 317–326.
- [39] S. Sueda, “Catmull-rom splines,” 2016, material de curso (CSC471), Texas A&M University. Accedido: 15 de junio de 2026. [Online]. Available: <https://people.engr.tamu.edu/sueda/courses/CSC471/2016S/demos/alling/index.html>
- [40] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, “Reciprocal n-body collision avoidance,” in *Robotics research: the 14th international symposium ISRR*. Springer, 2011, pp. 3–19.
- [41] Microsoft, “Object.GetHashCode method,” 2023, accedido: 15 de junio de 2026. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/api/system.object.GetHashCode>
- [42] G. Fowler, L. C. Noll, and P. Vo, “Fowler-noll-vo hash function,” 1991, accedido: 15 de junio de 2026. [Online]. Available: <http://www.isthe.com/chongo/tech/comp/fnv/>
- [43] Mercuna, “Mercuna 3d navigation,” 2026, consultado el 17 de junio de 2026. [Online]. Available: <https://mercuna.com/>
- [44] Epic Games, “Unreal engine,” <https://www.unrealengine.com/>, 2026, consultado el 17 de junio de 2026.
- [45] Mercuna, “Mercuna 3d navigation licensing,” 2026, consultado el 17 de junio de 2026. [Online]. Available: <https://mercuna.com/licensing/>
- [46] Mercuna Developments Ltd., “Any surface navigation UE user guide,” <https://mercuna.com/documentation/any-surface-navigation-ue-user-guide/>, consultado el 17 de junio de 2026.
- [47] Unity Technologies, “Unity Asset Store,” <https://assetstore.unity.com/>, 2026, consultado el 17 de junio de 2026.
- [48] Softbrix Studio, “Nav3D – unity asset store,” <https://assetstore.unity.com/packages/tools/behavior-ai/nav3d-100285>, consultado el 17 de junio de 2026.

- [49] Nav3D, “Nav3d,” 2026, consultado el 17 de junio de 2026. [Online]. Available: <https://nav3d.pro/>
- [50] —, “Nav3d | pathfinding & navigation for flying ai,” 2026, consultado el 17 de junio de 2026. [Online]. Available: <https://assetstore.unity.com/packages/tools/game-toolkits/nav3d-pathfinding-navigation-for-flying-ai-337874>
- [51] Infohazard Games, “Hypernav: 3d pathfinding and navigation,” 2026, consultado el 17 de junio de 2026. [Online]. Available: <https://infohazardgames.com/docs/Infohazard.HyperNav/html/>
- [52] Tec-Maid, “Spatial pathfinding - 3d a* algorithm,” 2026, consultado el 17 de junio de 2026. [Online]. Available: https://github.com/Tec-Maid/wiki/tree/master/Spatial_Pathfinding
- [53] Godot Engine Contributors, “Godot engine,” <https://godotengine.org/>, 2026, version [Insert Version Here].
- [54] T. Salmela, “Game development using the open-source godot game engine,” 2022.
- [55] R. S. Pressman, *Ingeniería del software: un enfoque práctico*, 7th ed. McGraw-Hill Interamericana, 2010.
- [56] Indeed, “Sueldo de un programador junior en españa,” 2026, consultado el 17 de junio de 2026. [Online]. Available: <https://es.indeed.com/career/programador-junior/salaries>
- [57] Van den Berg, Jur and Guy, Stephen J. and Snape, Jamie and Lin, Ming C. and Manocha, Dinesh, “RVO2-3D library: Reciprocal collision avoidance for real-time multi-agent simulation,” <https://gamma.cs.unc.edu/RVO2/>, 2016, consultado el 17 de junio de 2026.